

Plug-and-play sanitization for TFHE

Florian Bourse¹ and Malika Izabachene²

¹ Independent Scholar

² ETIS, UMR 8051 CY Cergy Paris Université, ENSEA, CNRS

Abstract. Fully Homomorphic encryption allows the evaluation of any circuits over encrypted data while preserving the privacy of the data. However, without any additional properties, no guarantee is provided for the privacy of the circuits which are evaluated.

A sanitization algorithm allows to destroy all previous information about how a ciphertext was obtained, ensuring that the circuit which was evaluated remains secret. In this paper, we present two new techniques to randomize RLWE ciphertexts, and show how they can be used to achieve ciphertext sanitization for the TFHE scheme proposed by Chillotti *et al.* (Asiacrypt 2016), by modifying the bootstrapping procedure internally.

Our first technique relies on a generalization of the strategy proposed by Bourse *et al.* (Crypto 2016) to the ring setting. Using a backward induction over the circuit size, we also improve on their proof technique to avoid randomization at each step of the computation, enabling faster randomization and smaller noise growth. While this first approach adapts well in theory, we show evidence that it fails to provide a practical solution and propose a second solution with more realistic parameters at the cost of using an additional public key.

As an additional contribution, we improve on the prohibitive size of the public key by relaxing the circuit privacy property to its computational counterpart, and build an efficient public randomizer composed of an RLWE-based public key encryption with additional properties on the ciphertexts distribution. We show that this randomizer can be used in the soak-and-spin paradigm of Ducas and Stehlé (Eurocrypt 2016) as well, and that it yields a significant improvement, mainly in the size of the public key.

As a proof of concept, we provide a C implementation of our sanitization strategy, which shows that a sanitized LWE ciphertext can be obtained almost for free compared to a bootstrapped LWE ciphertext assuming many discrete Gaussian samples at hand.

Keywords: Fully Homomorphic Encryption, circuit privacy, leftover hash lemma, sanitization

1 Introduction

A Fully Homomorphic Encryption (FHE) scheme allows to evaluate any circuit over encrypted data without having to decrypt them. A standard requirement for homomorphic encryption schemes is semantic security against chosen plaintext attacks, which guarantees that the data remains unknown to the party who evaluates the circuit.

For most of the known FHE schemes, semantic security relies on the hardness of the Learning With Errors (LWE) problem [Reg05], or variants of LWE, meaning that the message is slightly offset by some noise that grows after a homomorphic computation. In order to be able to continue the computation, a procedure called bootstrapping, first described in [Gen09], is used to refresh the noise in a ciphertext to a fixed manageable level.

The bootstrapping procedure is costly compared to basic operations, but packing techniques (e.g., [Bra12,BGV12,FV12,CKKS17,CZ17,MS18,GPvL23]) provide efficient amortized timings. Unfortunately, these techniques still have a very high latency (even though lots of ciphertexts can be bootstrapped at once, the time it takes to bootstrap a single

ciphertext remains prohibitively large) and are not always compatible with certain real-time applications. A series of works (e.g., [AP14, DM15, CHK⁺18, CGGI20]) tackles this issue by optimizing the bootstrapping of a single ciphertext, which is desirable in some scenarios.

Circuit privacy. Another property called circuit privacy is also crucial in many scenarios where the evaluation circuit contains sensitive information; typical examples being classification algorithms or financial prediction algorithms or many other scenarios where the delegated computation party may act as a service provider. Intuitively, the idea behind circuit privacy is that no one can reverse-engineer the computation into which a ciphertext went through. A bit more formally, an FHE scheme achieves circuit privacy for a class of functions if the output distribution of the evaluation procedure of the FHE scheme only depends on the result and does not leak information on which function from the class was evaluated. In addition, knowing the secret key should not give any advantage.

The property of circuit privacy has also found applications in cryptographic protocols: a first example is the computation of a private set intersection of two datasets based on homomorphic encryption [CLR17, CMdG⁺21] where a receiver has input set X and sends its dataset encrypted to a sender with input Y . The latter performs some homomorphic computation so that at the end, the receiver outputs $X \cap Y$. In order to provide security against a semi-honest receiver, one technique relies on using a circuit-private homomorphic encryption scheme. Another example can be found in the lattice-based threshold signature constructions [BGG⁺18, ASY22], built from threshold FHE, which also require to hide information in the computation of the partial decryption shares.

Different flavors of circuit privacy may be desired and some relations between them can be established. Circuit privacy against malicious adversaries, where the public keys and/or ciphertexts are not necessarily honestly generated, is much stronger than circuit privacy against passive adversaries. Fortunately, [OPP14] devised a technique to upgrade a compact FHE scheme circuit-private against passive adversaries using another (possibly non-compact) FHE scheme circuit-private against malicious adversaries, which can even possibly start from a non circuit-private compact FHE schemes using some additional twists. Also, depending on the application, the result of the computation might be sent back and decrypted directly, but also might be sent to another server for additional computation. This refinement of the circuit-private property has been defined in [GHV10] as 1-hop for the first case, or multi-hop for the second case. An i -hop circuit-private FHE scheme allows for i steps of computation before the ciphertext has to be decrypted. Finally, an FHE scheme could achieve circuit privacy for a particular class of functions but might fail to evaluate other functions in a circuit-private way. Another closely related property is sanitization of FHE ciphertexts as defined in [DS16], which asks that the FHE scheme includes a **Sanitize** algorithm which maps ciphertexts to a canonical distribution depending only on its underlying plaintext. It is straightforward to see that sanitization implies circuit privacy: appending **Sanitize** at the end of each evaluation procedure makes it possible for the FHE scheme to reach multi-hop circuit privacy for all functions. The other direction requires some additional assumptions and technical details such as circular security assumption, but intuitively, a circuit-private evaluation of the bootstrapping procedure would yield a sanitization algorithm. We also note that the notion of circuit privacy for FHE is very closely related to circuit privacy in multiparty computation (MPC), as FHE can be used as a building block in low communication MPC protocols.

1.1 Previous works

The first circuit-private technique was proposed in [Gen09] and relies on noise flooding by adding a fresh encryption of zero with a super-polynomially larger noise. Indeed, the noise contained in a ciphertext is the main source of leakage. And the idea is to add a noise with a very large parameter at the end of the computation to drown the existing noise and to prevent an adversary from making use of the information the noise contains. This yields a 1-hop scheme that is circuit-private for the class of all functions. The noise flooding technique requires the starting modulus-to-noise ratio to be super-polynomial, which makes the security rely on a stronger LWE assumption.

A new approach, proposed in [DS16] avoids this downside, and achieves circuit-privacy for FHE schemes relying on LWE with a polynomial modulus-to-noise ratio. Their sanitization algorithm is based on a paradigm called *soak and spin*, which consists of the iterations of two consecutive steps: a first one calls a **Refresh** algorithm which reduces the noise of the input ciphertext to a fixed level by invoking the bootstrapping; a second step, **ReRand** injects noise in the ciphertext in order to make it closer to a canonical distribution of ciphertexts. After λ consecutive iterations of **Refresh** and **ReRand** over any two input ciphertexts decrypting to the same message, the statistical distance between the two output ciphertexts is bounded by $2^{-\lambda}$. An additional requirement to their work is that the output of **ReRand** should decrypt to the correct message with very high probability in order to reach the sanitization property. As an application, any bootstrapping algorithm can be used as the **Refresh** algorithm, and they suggest to use a standard public key encryption consisting of a public key containing around $N \log q$ encryptions of 0 as **ReRand**, a random subset that is added to randomize, together with a random small enough noise. The authors of [AP20] proposed a refined security analysis of the sanitization algorithm from [DS16] based on the use of the Rényi divergence instead of the statistical distance. They showed that the number of iterations can be reduced depending on the target number of evaluations that needs to be hidden. They also set the parameters in the soak and spin process such that decryption fails with exponentially small probability.

In [BdPMW16], Bourse *et al.* showed circuit privacy for NC^1 circuits without relying on the circular security assumption, and basing its security on plain LWE with polynomial modulus-to-noise ratio. The main idea of their approach is to tweak the computation paradigm to branching programs, which are shown to be able to be evaluated in a circuit-private manner using a two steps randomization process at each stage of the computation. The randomization process consists of both switching the homomorphic product to its randomized counterpart from [AP14], and adding a random Gaussian noise after each product. The first step ensures that the distribution of the noise after one step of computation is close to a Gaussian noise, with a parameter independent of the previous states of the evaluation; this is done by making use of the randomized homomorphic product. Adding a random Gaussian noise in a second step ensures that the noise does not leak information about the computation being carried on. The downside of this work is that it applies to schemes that are not implemented in practice for efficiency reasons, namely the GSW cryptosystem [GSW13] and its variants [BV14], and requires the functions to be converted to branching programs.

Concurrent works. [Klu24] introduced a statistical LWE sanitization procedure by modifying the FHEW-like bootstrapping restricted to binary secret keys. Though statistical sanitization is stronger, we investigate other trade-offs by proposing a second sanitization strategy that relies on RLWE, a computational assumption. With this relaxation, the

sanitization property holds under the same assumption as the semantic security does, allowing us to considerably reduce the size of the public key. For sake of comparison, in [Klu24], the public sanitization key is composed of $2^{11.5}$ to $2^{12.3}$ fresh LWE samples of zero in dimension $N = 2^{11}$, while in our case it is made of one RLWE encryption of zero in dimension $N = 2^{11}$.

Recently, [HMS25] proposed a more efficient sanitization procedure for TFHE than ours. Their solution builds upon our idea of using a public key randomizer and also relies on the (computational) RLWE assumption. Their proof strategy for the randomizer follows the same blueprint as ours, except that they use rounded Gaussian samples instead of discrete Gaussians. One of the main differences with our approach lies in the use of randomization before and after bootstrapping, rather than internally within the bootstrapping process. This technique leverages the homomorphic rounding operation to eliminate the information leaked through the noise of the output ciphertext. In contrast, our method introduces slight internal modification to the bootstrapping procedure, and the proof relies on a generalization of the noise analysis from [BdPMW16] to the ring setting. This allows us to achieve multi-hop circuit privacy for branching programs over rings without relying on bootstrapping as opposed to [Klu24,HMS25] and, consequently, without requiring the circular security assumption.

1.2 Our contributions

We propose a practical sanitization approach for FHE schemes with polynomial modulus-to-noise ratio. Our solution is based on the bootstrapping procedure implemented using GSW homomorphic product and applies to TFHE. As explicitly illustrated in [MP21], the TFHE and FHEW bootstrappings are close and our strategy can adapt to the setting of FHEW as well. However, in FHEW bootstrapping, only certain components of the bootstrapping key are used, where the selection depends on the input ciphertext. To conceal which components were actually used, our strategy would require adding dummy computations, leading to a significant drop in efficiency. In contrast, TFHE bootstrapping used all components of the bootstrapping key, eliminating the need to hide which components of the key were used.

Also, our sanitization algorithm is compatible with recent improved functional bootstrapping [BDF18,CIM19,GBA21,CLOT21,LY23] which allows to support larger space messages with higher precision, provided the parameters are adapted. As a direct application of our sanitization result, we additionally obtain multi-hop circuit privacy for the class of all functions.

One of our building blocks is the generalization of the randomization techniques from [AP14,BdPMW16] to the ring setting, adapting techniques from [MM11] to cyclotomic rings with power-of-two modulus (see Lemma 7). Plugging our results into [BdPMW16] gives us circuit-privacy for the class NC^1 (more precisely, for the evaluation of branching programs of polynomial size) for the ring variant of GSW without bootstrapping, allowing e.g. packing messages. We also propose a new induction proof which slightly improves parameters by avoiding rerandomization at every step of the branching program evaluation.

Our second main tool is a new technique to rerandomize RLWE ciphertexts when the parameters are too small for the Leftover Hash Lemma to apply. We introduce a simple RLWE public key encryption scheme and show that the distribution of the ciphertexts behaves nicely, even though the adversary knows the secret key. This allows us to considerably reduce the size of the public key required to reach sanitization, at the cost of replacing the statistical statement for sanitization with its computational counterpart.

The sanitization property now relies on the same RLWE assumption as the one used for assessing the semantic security of the FHE scheme.

In order to measure the efficiency of our scheme, we implemented a proof of concept in C. Our experiments show that most of the time is spent on generating Gaussian samples over the integers. As we believe a lot of applications can either perform this step in parallel, during the downtime of the server, or via specialized hardware, we also measured the efficiency of our strategy assuming infinitely many Gaussian samples at hand. Our experiment result shows that the overhead induced by the sanitization is very small compared to the cost of the non-sanitizing TFHE bootstrapping for the same set of parameters.

For the sake of completeness, we also discuss the instantiation of [DS16] with our new rerandomization algorithm, and provide a detailed comparison with our approach in Appendix A.

Overview of the techniques. The underlying idea of our high-level strategy is based on two observations. Recall that the bootstrapping procedure of TFHE can be decomposed in 3 steps, called **BlindRotate**, **Extract**, and **KeySwitch**, as illustrated on Figure 1. So if any of these 3 steps destroys unwanted information of the input, all the information about any previous evaluation on the ciphertext is washed away, and the bootstrapping procedure becomes a sanitizing algorithm.

Our focus will be on the **BlindRotate** procedure which we carefully modify to be a circuit private evaluation for the class of function $\text{BlindRotate}_{\mathbf{a},b} : \mathbf{s} \mapsto (0, \text{testv}) \cdot X^{b - \sum_i s_i a_i}$ parameterized by a LWE ciphertext $(\mathbf{a}, b)$ for any polynomial testv and a variable $\mathbf{s} = (s_0, \dots, s_{n-1}) \in \{0, 1\}^n$. We notice that the **BlindRotate** computation is mostly made of homomorphic evaluations of multiplexers i.e. conditional operators, which behave nicely with respect to the circuit privacy property as shown in [BdPMW16]³. We modify the **BlindRotate** procedure to make it circuit-private, by randomizing the gadget decomposition that we combine with a fresh randomizer at the end of the computation.

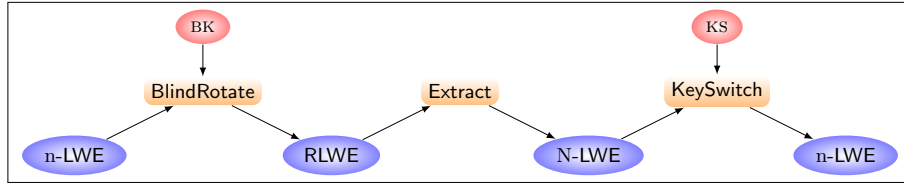


Fig. 1. TFHE bootstrapping steps: n-LWE denotes a LWE ciphertext of dimension n . After an initialization step not pictured here, the bootstrapping of an n-LWE ciphertext into a fresh n-LWE ciphertext goes through three algorithms **BlindRotate**, **Extract**, and **KeySwitch** and requires two sets of keys: the bootstrapping key BK and the keyswitching key KS. More details will be given later in the paper.

To avoid some of the dependencies between random variables, [BdPMW16] proceeds by induction and analyzes the distribution of the ciphertext after each step of the computation, starting from the initial state. In order to prove their result, a small Gaussian noise is required to be added at each step of the computation. We improve on their strategy by using another proof by induction, starting from the end towards the beginning, analyzing the noise added by the remainder of the computation, and we show that adding just one public randomizer at the very end is enough to prove circuit private evaluation of

³ [BdPMW16] shows circuit-private evaluation for branching programs, while not exactly being the same, it is close to multiplexers evaluation.

BlindRotate and hence FHE ciphertexts sanitization. We notice that this proof technique can also be applied to their setting as well and improve the parameters.

Additional challenges have to be tackled as **BlindRotate** is not exactly a branching program. An additional difficulty compared to the evaluation of branching programs is that the computation does not just carry information through the different stages of the evaluation, but there are also multiplications by powers of X being done on the underlying plaintext, and also on the previous states of the accumulator inside the bootstrapping. In addition, as we are dealing with polynomials and not scalar vectors, which means that the different components are combined with each other along products, the standard tools at hand for analyzing distribution cannot apply directly because of the dependencies between components. To tackle these issues, we carefully craft distributions that are spherical so that multiplying them by a power of X doesn't change the distribution. We then explicitly state the matrix-vector products along polynomial multiplications in order to exploit known results on lattices.

To give a bit more details on one step of the induction, the distribution of **LWE** ciphertexts is given by two elements beside its associated plaintext: a vector that should be sampled uniformly at random, and the noise component and both parts need to be considered by our analysis. As in [BdPMW16], we leverage the fact that the decomposition is now randomized with a large enough parameter in order to prove that the noise is statistically close to a discrete Gaussian distribution as a linear combination of discrete Gaussian distributions. And we add a small Gaussian variable that ensures that the support of this discrete Gaussian is not a sublattice, i.e., all the values can be reached by this noise and it is not restricted to a subset of values that could leak information about the evaluation being carried on. As a side contribution, we generalize the noise analysis from [BdPMW16] to the ring setting and to arbitrary lattices. We also extend underlying lemmata and derive underlying bounds to handle Gaussian parameters that are not necessarily spherical by viewing products of polynomials in our specific case as matrix-vector products.

We also specialize the Leftover Hash Lemma to our setting, by analyzing the structure of the ring, in order to randomize the uniform random vector part. However, this requires quite large vectors, especially since we consider a power-of-two modulus that is not prime. For example, when half of the elements are not invertible (which is the case both with integers modulo a power of 2 and with polynomials modulo $X^N + 1$ modulo a power of 2), even when sampling two uniformly random vectors $\mathbf{a}$ and $\mathbf{b}$ of size m , their inner-product can't be statistically close to uniform unless m is at least super-logarithmic in the security parameter λ . This is because with probability $\frac{1}{2^m}$, none of the elements of $\mathbf{a}$ is invertible in such a ring, so the result is not invertible. The bound we derive on the parameters with this technique was not practical. The full conditions are given in Lemma 9 of Section 3. In **TFHE**, the parameters are optimized for efficient bootstrapped operations, and in that case, the polynomial vectors could be of size at most 6 and of size 7. So this first approach cannot be used for a practical implementation of circuit privacy or sanitization. A possible strategy would be to make use of an additional public key encryptions of zero as suggested in [DS16]. However, as this technique relies on the Leftover Hash Lemma, it requires many encryptions of zero i.e. $N \log q$, where N is the **LWE** dimension and q is the modulus.

In Section 3.4, we propose a more efficient strategy that makes use of a suitable public key encryption scheme for which we characterize the requisites to reach FHE ciphertexts sanitization. In practice, the public randomizer just consists of a single encryption of zero, which is multiplied by a fresh Gaussian vector, and can be computed very efficiently. In

this case, the sanitization property is relaxed to its computational counterpart, relying on the hardness of the decision RLWE problem with not necessarily binary secret key distribution. This construction is inspired by the techniques used in the context of Functional Encryption in [MKMS22] where one needs to protect its data against an adversary that can partially decrypt the ciphertext. We believe that this result is of independent interest. In particular, it can be plugged directly together with the technique of [DS16] to improve its efficiency at the cost of relaxing the sanitization property to hold computationally.

Organization. In Section 2, we present the notations used through the paper and recall the preliminaries about Gaussian distributions and the TFHE scheme. In Section 3, we present our generalization of the randomization techniques of [BdPMW16] that will be used for our result, as well as our new public key encryption scheme that verifies the required properties. In Section 4, we present our techniques for circuit privacy and show how to build a sanitization algorithm for TFHE. In Section 5, we discuss the practical parameters of our implementation.

2 Preliminaries

In this section, we give notations, mathematical definitions and lemmata we will use for our proofs.

2.1 Notations and definitions

In this paper, we will note λ a security parameter. The set of integers from 1 to n will be noted $[1, n]$ for convenience. We use lower case bold font, e.g. $\mathbf{a}$, to denote (possibly row) vectors, and upper case bold font, e.g. $\mathbf{A}$, to denote matrices. We write the left-right concatenation of matrices using $|$, e.g. $(\mathbf{A} \mid \mathbf{B})$. We will use $\otimes$ to denote the Kronecker product of two matrices. We let q be an integer modulus such that $q = B^\ell$ with B a power of 2. We denote $\mathbb{T}$ the set of real numbers modulo one and the discretized torus $\hat{\mathbb{Z}}_q = \{0, \frac{1}{q}, \dots, \frac{q-1}{q}\}$ is $\frac{1}{q}\mathbb{Z} \cap \mathbb{T}$. Note that $\hat{\mathbb{Z}}_q$ is isomorphic to $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$. In particular, the multiplication over $\hat{\mathbb{Z}}_q$ is given by $x \cdot y = qx \cdot y$, for all $x, y \in \hat{\mathbb{Z}}_q$.

We set $\mathbb{B} = \{0, 1\}$ and $\mathbb{B}_N(X)$ the set of polynomials of degree at most N with coefficients in $\mathbb{B}$. We denote $R = \mathbb{Z}[X] \bmod X^N + 1$ the set of integer polynomials of degree at most N , where N is a power-of-two. We set $R_q = \mathbb{Z}_q[X] \bmod X^N + 1$ and $\hat{R}_q$ the set of polynomials with coefficients over $\hat{\mathbb{Z}}_q$ modulo $X^N + 1$. Vectors are denoted as row vectors, and $\mathbf{a}^t$ denotes the column vector which is the corresponding transpose vector of $\mathbf{a}$. The absolute value of an element a in $\mathbb{T}$, denoted $|a|$, is the absolute value of its representative that is closest to 0 (i.e., in $] -0.5, 0.5[$). We also use this representative when we define norms for vectors, matrices, and polynomials, and also use those for elements in $\hat{\mathbb{Z}}_q$. The euclidean norm of a vector $\mathbf{a}$ is denoted $\|\mathbf{a}\|_2$ and its infinity norm is denoted $\|\mathbf{a}\|_\infty$. The spectral norm of a square matrix $\mathbf{A}$, i.e. its largest singular value, is denoted $\|\mathbf{A}\|_2$. In order to avoid cumbersome notations throughout the paper, we will use the following shorthand notation to define anticyclic matrices corresponding to polynomial multiplications modulo $X^N + 1$. We denote $\text{pow}_X = (1, X, \dots, X^{N-1}) \in R^{1 \times N}$.

For any polynomial $p = \sum_{i=0}^{N-1} p_i X^i \in \hat{R}_q$, there exists a matrix $\mathbf{P} = \begin{pmatrix} p_0 & -p_{N-1} & \dots & \dots & -p_1 \\ p_1 & p_0 & -p_{N-1} & \dots & -p_2 \\ \vdots & & \ddots & & \vdots \\ p_{N-1} & \dots & \dots & p_1 & p_0 \end{pmatrix} \in \hat{\mathbb{Z}}_q^{N \times N}$ such that $\text{pow}_X \cdot \mathbf{P} = \text{pow}_X \otimes p$. We also use the

following euclidean norm for polynomials in $\hat{R}_q$: $\|p\|_2 = \sqrt{\sum_{i=0}^{N-1} p_i^2}$. Note that the spectral norm of a matrix $\mathbf{P}$ is upper bounded by $\sqrt{N} \|p\|_2$.

Gadget vector. We let $\mathbf{g} = (1/B, \dots, 1/B^\ell) \in \hat{R}_q^{1 \times \ell}$ be the vector of powers of B and define the gadget matrix as $\mathbf{G} = \mathbf{I}_{d+1} \otimes \mathbf{g}^t$, i.e.

$$\mathbf{G}^t = \begin{pmatrix} 1/B \dots 1/B^\ell & 0 & \dots & 0 & \dots & 0 \\ 0 & \dots & 0 & 1/B \dots 1/B^\ell & \ddots & \vdots \\ \vdots & & \ddots & & \ddots & 0 \\ 0 & \dots & 0 & \dots & 0 & 1/B \dots 1/B^\ell \end{pmatrix} \in \hat{R}_q^{(d+1) \cdot \ell \times (d+1)} \quad (1)$$

2.2 Random variables

Variance and covariance. For a random variable X , we denote $E[X]$ the expected value of X . The covariance of two random variables X and Y is $E[(X - E[X])(Y - E[Y])]$. The covariance matrix $\text{Var}(\mathbf{X})$ of a vector of random variables $\mathbf{X} = (X_1, \dots, X_N)$ is the matrix whose coefficient on row i , column j is the covariance of X_i and X_j . For any two real vectors of independent random variables $\mathbf{X}$ and $\mathbf{Y}$ and any real α , we have: $\text{Var}(\alpha\mathbf{X} + \mathbf{Y}) = \alpha^2 \text{Var}(\mathbf{X}) + \text{Var}(\mathbf{Y})$.

Statistical distance and min-entropy. We write $\mathbf{y} \leftarrow \mathcal{P}$ when y is sampled from distribution $\mathcal{P}$. The statistical distance between two probability distributions $\mathcal{P}$ and $\mathcal{Q}$ over a discrete domain X is defined as:

$$\Delta(\mathcal{P}, \mathcal{Q}) = \frac{1}{2} \sum_{a \in X} |\mathcal{P}(a) - \mathcal{Q}(a)|$$

We say that two distributions $\mathcal{P}$ and $\mathcal{Q}$ are (statistically) close and we write $\mathcal{P} \approx_s \mathcal{Q}$ if their statistical distance is negligible in λ . We say that they are computationally close and we write $\mathcal{P} \approx_c \mathcal{Q}$ if no algorithm can distinguish between the two distributions in polynomial time with non-negligible probability, i.e. for any polynomial time algorithm $\mathcal{A}$, $\mathcal{A}(\mathcal{P}) \approx_s \mathcal{A}(\mathcal{Q})$. The min-entropy of a random variable X is defined as $H_\infty(X) = -\log \max_x \Pr[X = x]$. It gives a bound on the probability of collision for two independent identically distributed random variables X and X' : $\Pr(X = X') \leq 2^{-H_\infty(X)}$. The min-entropy is a useful tool when analyzing conditional probability distribution thanks to the following property: for any random variable Y on a set $\mathcal{Y}$,

$$H_\infty(X | Y) \geq H_\infty(X) - \log |\mathcal{Y}|.$$

2.3 Gaussian distribution over lattices

Lattices. An m -dimensional lattice is a discrete subgroup of $\mathbb{R}^m$. Given k linearly independent vectors of $\mathbb{R}^m$ ($\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_k$), the lattice generated by $\mathbf{B} = (\mathbf{b}_1 \mid \mathbf{b}_2 \mid \dots \mid \mathbf{b}_k)$ is of rank k and is denoted:

$$\Lambda(\mathbf{B}) = \left\{ \sum_{i=1}^k \mathbf{x}_i \mathbf{b}_i \in \mathbb{R}^m \mid \mathbf{x}_i \in \mathbb{Z} \right\}.$$

For any $\mathbf{v} \in \hat{\mathbb{Z}}_q^{d+1}$, we denote the cosets of the lattice orthogonal to the gadget matrix $\mathbf{G}$ defined in equation (1) by

$$\Lambda_{\mathbf{v}}^{\perp}(\mathbf{G}) = \{\mathbf{u} \in \mathbb{Z}^{(d+1) \cdot \ell} \mid \mathbf{G}^t \mathbf{u} \in \mathbf{v} + \mathbb{Z}^{d+1}\}, \quad \Lambda^{\perp}(\mathbf{G}) = \Lambda_{\mathbf{0}}^{\perp}(\mathbf{G})$$

For the sake of readability, we also extend the notation $\Lambda^{\perp}(\mathbf{G})$ to the set of vectors of polynomials, by identifying a polynomial $u = \sum_{i=0}^{N-1} u_i X^i$ with the vector of its coefficients $\mathbf{u} = (u_0 \mid u_1 \mid \dots \mid u_{N-1})$:

$$\Lambda^{\perp}(\mathbf{G}) = \{\mathbf{u} \in R^{(d+1) \cdot \ell} \mid \mathbf{G}^t \mathbf{u} \in R^{d+1}\}$$

which is a full rank lattice of dimension $N(d+1) \cdot \ell$.

Gaussian Distributions. The *ellipsoidal Gaussian distribution* over $\mathbb{R}^n$ centered at $\mathbf{0}$ with covariance matrix $\Sigma = S^t S$ where $S \in \mathbb{R}^{m \times n}$ is a rank- n matrix, is defined as:

$$\rho_S(\mathbf{x}) = \exp(-\pi \mathbf{x}^t (S^t S)^{-1} \mathbf{x})$$

When $S = s\mathbf{I}_n$, the spherical Gaussian distribution ρ_S is also denoted ρ_s . The *ellipsoidal Gaussian distribution* with parameter S over a countable set C (a lattice Λ or a coset $\Lambda + \mathbf{x}$) is defined as $\forall \mathbf{x} \in C$, $\mathcal{D}_{C,S}(\mathbf{x}) = \frac{\rho_S(\mathbf{x})}{\rho_S(C)}$.

Lemma 1 (Preimage sampling [GPV08, Pei10, MP12, AP14]). *There is an efficient randomized decomposition function which on input $\mathbf{v} \in \hat{\mathbb{Z}}_q^{d+1}$ outputs a sample $\mathbf{u} \in \mathbb{Z}^{(d+1)\ell}$ from a distribution negligibly close to $\mathcal{D}_{\Lambda_{\mathbf{v}}^{\perp}(\mathbf{G}), \gamma}$ with parameter $\gamma = \tilde{O}(1)$.*

In addition, we can reach the target distribution exactly using the sampler from [BLP⁺13].

2.4 Additional lemmata

In the following, we will denote $\eta_{\epsilon}(\Lambda)$ the smoothing parameter of a lattice Λ . Intuitively, it is a Gaussian parameter value beyond which discrete Gaussian distributions over Λ behave almost as continuous Gaussian distributions, ϵ being a bound on the statistical distance that appears in the following lemmata. The next result gives a bound on the smoothing parameter of a generic lattice.

Lemma 2 ([MR07, Lemma 3.3]). *Let Λ be any rank- m lattice and ϵ be any positive real. Then*

$$\eta_{\epsilon}(\Lambda) \leq \lambda_m(\Lambda) \cdot \sqrt{\frac{\ln(2m(1+1/\epsilon))}{\pi}}$$

where $\lambda_m(\Lambda)$ is the smallest R such that the ball $\mathcal{B}_R$ centered in the origin and with radius R contains m linearly independent vectors of Λ .

Lemma 3 ([Ban93]). *For any n -dimensional lattice Λ and parameter $s > 0$, the euclidean norm of a sample $\mathbf{u}$ from $\mathcal{D}_{\Lambda, s}$, $\|\mathbf{u}\|_2 \leq s\sqrt{n}$, except with probability at most 2^{-2n} .*

Lemma 4 ([AGHS13, Lemma 4]). *For any rank- m lattice Λ , $0 < \varepsilon < 1$, vector $\mathbf{c} \in \mathbb{R}^m$, and rank- m matrix $S \in \mathbb{R}^{k \times m}$, such that $\sigma_m(S) \geq \eta_\varepsilon(\Lambda)$, we have:*

$$\rho_S(\Lambda + \mathbf{c}) \in \left[\frac{1-\varepsilon}{1+\varepsilon}, 1 \right] \cdot \rho_S(\Lambda).$$

where $\sigma_m(S)$ is the smallest singular value of S .

Lemma 5 (Simplified version of [Pei10, Theorem 3.1]). *For any rank- m lattice Λ , $0 < \varepsilon < \frac{1}{2}$, vector $\mathbf{c} \in \mathbb{R}^m$, and rank- m matrix $S_1 \in \mathbb{R}^{k_1 \times m}$ and $S_2 \in \mathbb{R}^{k_2 \times m}$. If*

$$\min \left(\sigma_m \left(\sqrt{S_1^t S_1 + S_2^t S_2} \right), \sigma_m \left(\sqrt{\left((S_1^t S_1)^{-1} + (S_2^t S_2)^{-1} \right)^{-1}} \right) \right) \geq \eta_\varepsilon(\Lambda),$$

$$\Delta(\mathbf{y}_1 + \mathbf{y}_2, \mathbf{y}') \leq 8\varepsilon$$

where $\mathbf{y}_1 \leftarrow \mathcal{D}_{\Lambda+\mathbf{c}, S_1}$, $\mathbf{y}_2 \leftarrow \mathcal{D}_{\Lambda, S_2}$, and $\mathbf{y}' \leftarrow \mathcal{D}_{\Lambda+\mathbf{c}, \sqrt{S_1^t S_1 + S_2^t S_2}}$

Lemma 6 ([KLSS23, Lemma 5]). *For any rank- m lattice Λ , $\varepsilon > 0$ and rank- m matrix $S \in \mathbb{R}^{k \times m}$, $\sigma_m(S) \geq \eta_\varepsilon(\Lambda)$ if $\left\| (S^t S)^{-1} \right\|_2 \leq \eta_\varepsilon(\Lambda)^{-2}$.*

2.5 Fully Homomorphic Encryption

In this paper, we focus on secret key fully homomorphic encryption for efficiency reasons, meaning that the same key is required for encryption and decryption. This is sufficient for many applications, our results can still be easily generalized to the public key setting if required, by adding an encryption key which consists of a set of encryptions of zero. The main downside then becomes storing the public key.

Definition 1 (Fully Homomorphic Encryption). *A fully homomorphic encryption scheme is given by four polynomial time algorithms, (KeyGen, Encrypt, Decrypt, Eval) described as follow:*

KeyGen(1^λ) *on input a security parameter λ outputs an evaluation key EVK and a secret key SK;*

Encrypt(SK, μ) *on input a secret key SK and a message μ returns a ciphertext CT;*

Decrypt(SK, CT) *on input a secret key SK and a ciphertext CT returns a message μ ;*

Eval(EVK, f , CT₁, ..., CT_t) *on input an evaluation key EVK, a function f on t inputs, and t ciphertexts CT₁, ..., CT_t returns a ciphertext CT_f.*

Let us denote $\mathcal{M}$ the message space and $\mathcal{C}$ the ciphertext space. For $\mu \in \mathcal{M}$, we define $\mathcal{C}_\mu = \text{Decrypt}(\text{SK}, \cdot)^{-1}(\mu)$, the set of all ciphertexts that decrypt to μ .

We say that an FHE scheme is *correct* if, for (EVK, SK) sampled from **KeyGen**(1^λ) :

- for all messages $\mu \in \mathcal{M}$: **Encrypt**(SK, μ) $\in \mathcal{C}_\mu$ with overwhelming probability.
- for all functions $f : \mathcal{M}^t \rightarrow \mathcal{M}$, $(\mu_1, \dots, \mu_t) \in \mathcal{M}^t$, $(\text{CT}_1, \dots, \text{CT}_t) \in \mathcal{C}_{\mu_1} \times \dots \times \mathcal{C}_{\mu_t}$: **Eval**(EVK, f , CT₁, ..., CT_t) $\in \mathcal{C}_{f(\mu_1, \dots, \mu_t)}$ with overwhelming probability;

We say that an FHE scheme is *compact* if the ciphertexts are of polynomial size. We say that an FHE has *indistinguishability under chosen plaintext attacks* (IND-CPA security) or is *semantically secure* if no polynomial time adversary can have a non-negligible advantage in guessing a bit β given oracle access to the function $(\mu_0, \mu_1) \mapsto \text{Encrypt}(\text{SK}, \mu_\beta)$.

Definition 2 (Circuit Privacy). A (fully) homomorphic encryption scheme is circuit-private for a class of functions $\mathcal{F}$ if the homomorphic evaluation of a function $f \in \mathcal{F}$ on encrypted messages does not leak more information than the evaluation result, even given the secret key, i.e. there exists a polynomial time simulator Sim such that the following property holds for any $f \in \mathcal{F}$:

$$\begin{aligned} & (\text{Sim}(1^\lambda, f(\mu_1, \dots, \mu_t), (\text{CT}_1, \dots, \text{CT}_t), \text{EVK}), (\text{CT}_1, \dots, \text{CT}_t), \text{SK}) \\ & \approx_s (\text{Eval}(\text{EVK}, f, (\text{CT}_1, \dots, \text{CT}_t)), (\text{CT}_1, \dots, \text{CT}_t), \text{SK}), \end{aligned}$$

where $\text{CT}_i \leftarrow \text{Encrypt}(\text{SK}, \mu_i)$, $(\text{EVK}, \text{SK}) \leftarrow \text{KeyGen}(1^\lambda)$.

Remark 1. The definition above considers the joint probability distribution by including the secret key SK , which means that a distinguisher can use it to build an attack. Some of the previous definitions of circuit privacy considers the distribution of the ciphertexts only but in that case, the statistical distance has to be small for any secret key SK . In that case, the value of SK can be hardwired into the distinguisher, so both definitions prove the circuit privacy security property against an adversary knowing the secret key. However, proving security for any SK means that the statistical distance is small (knowing the secret key) even for maliciously generated secret keys. All our lemmata actually prove statements for any fixed SK which is stronger and also use lighter notations. However, our results rely on the randomness honestly generated in the ciphertexts $(\text{CT}_1, \dots, \text{CT}_t)$ or in the evaluation key EVK , so on the random coins of KeyGen .

Remark 2. In this work, we deal with honest-but-curious adversaries i.e. ciphertexts and keys are sampled correctly. To prevent attacks in the malicious setting, one can use the techniques of [OPP14] to upgrade our scheme, using a maliciously circuit-private (possibly non-compact) FHE scheme. In comparison, in the following definition of sanitization, we still assume that the keys are sampled correctly, however privacy holds for any ciphertext, even maliciously generated ones.

Sanitization of ciphertexts. Intuitively, the goal of a sanitization algorithm is to remove all information conveyed in the ciphertext beside the message, that is, we want to erase its memory from any previous evaluations. More formally, we ask that a sanitization algorithm verifies two properties: it must be message-space preserving and sanitizing. These properties are defined as follow:

$\text{Sanitize}(\text{EVK}, \text{CT})$ on input an evaluation key EVK and a ciphertext CT returns a ciphertext CT_s .

We say that Sanitize is *message-space preserving* if for any $\mu \in \mathcal{M}$, any ciphertext $\text{CT} \in \mathcal{C}_\mu$, $\text{Sanitize}(\text{EVK}, \text{CT}) \in \mathcal{C}_\mu$ with overwhelming probability. We say that Sanitize is *sanitizing* if there exists a polynomial time simulator Sim such that for any $\mu \in \mathcal{M}$, any ciphertext $\text{CT} \in \mathcal{C}_\mu$, the following holds:

$$(\text{Sim}(1^\lambda, \mu, \text{EVK}), \text{SK}) \approx_s (\text{Sanitize}(\text{EVK}, \text{CT}), \text{SK}),$$

where EVK is sampled honestly.

Remark 3. Note that given a sanitizing and message-space preserving Sanitize algorithm, it is easy to construct an FHE scheme that is circuit-private for all functions, by running Sanitize at the end of the evaluation procedure.

In this work, we also consider the relaxed variants of circuit-privacy and sanitization, where the closeness holds only computationally, instead of statistically.

2.6 Background on TFHE

The TFHE encryption scheme [CGGI20] encrypts messages in a subset $\mathcal{M}$ of $\mathbb{T}$, using LWE. For simplicity, we will take $\mathcal{M} = \{0, \frac{1}{2}\}$ in the following.

A LWE encryption of a message $\mu \in \mathcal{M}$ is a vector $(\mathbf{a} \mid \mathbf{a} \cdot \mathbf{s} + \mu + e) \in \hat{\mathbb{Z}}_q^{1 \times (n+1)}$ where $\mathbf{a}$ is uniform over $\hat{\mathbb{Z}}_q^{1 \times n}$, e is sampled from χ_ϑ — a noise distribution with variance ϑ . The secret key $\mathbf{s}$ is sampled from a uniform distribution over $\mathbb{B}^n$. In order to decrypt $(\mathbf{a} \mid b)$, one computes an approximation of μ defined as the phase of ciphertext $(\mathbf{a} \mid b)$, $\varphi_{\mathbf{s}}(\mathbf{a} \mid b) = b - \mathbf{a} \cdot \mathbf{s}$, and rounds it to nearest element in $\mathcal{M}$.

The first step of the bootstrapping is to deterministically map ciphertexts from $\hat{\mathbb{Z}}_q^{1 \times (n+1)}$ to $\mathbb{Z}_{2N}^{n+1}$. This incurs some rounding errors that could change the result of decryption on ill-formed ciphertexts. We thus take this additional rounding step into account when defining $\mathcal{C}_\mu$ in order to capture sanitization and message-space preserving even for maliciously generated ciphertexts. If parameters are set carefully, this has no impact on ciphertexts obtained in an honest way. For $\mu \in \{0, \frac{1}{2}\}$, we say that

$$\mathcal{C}_\mu = \left\{ (\mathbf{a} \mid b) \in \hat{\mathbb{Z}}_q^{1 \times (n+1)} \mid \left| [2Nb] - \sum_{i=1}^n s_i [2Na_i] - \mu \right| \leq \frac{1}{4} \right\}.$$

The error of a ciphertext $\mathbf{c} \in \mathcal{C}_\mu$ is defined as $\varphi_{\mathbf{s}}(\mathbf{c}) - \mu$, and its variance $\text{Var}(\mathbf{c})$ as the variance of the error of $\mathbf{c}$ or equivalently the variance of $\varphi_{\mathbf{s}}(\mathbf{c})$.

The semantic security of this encryption scheme relies on the hardness of the decision LWE problem [Reg05], or equivalently of its search counterpart:

- The decision LWE problem, parameterized by n and χ_ϑ , asks to distinguish the uniform distribution over $\hat{\mathbb{Z}}_q^{1 \times (n+1)}$ from the distribution of fresh LWE encryptions of 0 using the same secret key $\mathbf{s}$ sampled uniformly at random in $\mathbb{B}^n$.
- The search LWE problem asks to find $\mathbf{s}$ from polynomially many fresh LWE encryptions of 0 using the same secret key $\mathbf{s}$ sampled uniformly at random in $\mathbb{B}^n$.

Our sanitization algorithm is based on the bootstrapping procedure of TFHE, which is an optimized version of FHEW bootstrapping [DM15]. The idea behind FHEW-based bootstrapping is to compute a ciphertext of $X^{\varphi_{\mathbf{s}}(\mathbf{c})}$. In order to achieve this, polynomials are encrypted using two distinct schemes: RLWE — a variant of LWE using ring elements — and RGSW.

A RLWE encryption of a message $\mu \in \hat{R}_q$ is a vector $(\mathbf{a} \mid \mathbf{a} \cdot \tilde{\mathbf{s}} + \mu + e) \in \hat{R}_q^{1 \times (d+1)}$, where $\mathbf{a}$ is uniformly random in $\hat{R}_q^d$, e has coefficients sampled from χ_ϑ — ϑ is the noise variance. The secret key $\tilde{\mathbf{s}}$ is sampled from χ_ϑ^d . We denote $\text{RLWE}_{\tilde{\mathbf{s}}}(\mu)$ the set of random variables $(\mathbf{a} \mid b)$ in $\hat{R}_q^{1 \times (d+1)}$ whose phases $\varphi_{\tilde{\mathbf{s}}}(\mathbf{a} \mid b) = b - \mathbf{a} \cdot \tilde{\mathbf{s}}$ of center μ , and $\text{RLWE}_{\tilde{\mathbf{s}}, \vartheta}(\mu)$ a fresh RLWE encryption of μ under secret key $\tilde{\mathbf{s}}$ with noise variance ϑ . For $\mathbf{c} \in \text{RLWE}_{\tilde{\mathbf{s}}}(\mu)$, we define its error $\text{Err}(\mathbf{c}) = \varphi_{\tilde{\mathbf{s}}}(\mathbf{c}) - \mu$, and we denote $\text{Var}(\mathbf{c})$ the covariance matrix of its coefficients.

The semantic security of this encryption scheme relies on the hardness assumption of the RLWE decision problem, the ring variant of the LWE decision problem [SSTX09, LPR10].

A RGSW encryption of a message $\mu \in R$ is a matrix $\mathbf{Z} + \mu \mathbf{G}$ in $\hat{R}_q^{(d+1) \cdot \ell \times (d+1)}$, where each of the $(d+1) \cdot \ell$ rows of $\mathbf{Z}$ is a RLWE encryption of 0. The secret key is the same as for RLWE and the matrix $\mathbf{G}$ is defined as in Section 2.1. We denote $\text{RGSW}_{\tilde{\mathbf{s}}, \vartheta}(\mu)$ a fresh RGSW encryption of μ under secret key $\tilde{\mathbf{s}}$ with noise variance ϑ . More visually, we have:

$$\text{RGSW}_{\tilde{s},\vartheta}(\mu) = \begin{pmatrix} \text{RLWE}_{\tilde{s},\vartheta}(0) \\ \text{RLWE}_{\tilde{s},\vartheta}(0) \\ \vdots \\ \text{RLWE}_{\tilde{s},\vartheta}(0) \end{pmatrix} + \mu \mathbf{G} = (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) + \mu \mathbf{G}$$

where $\mathbf{A}$ is sampled uniformly at random in $\hat{R}_q^{(d+1) \cdot \ell \times d}$, and each coefficient appearing in $\mathbf{e}$ is sampled from χ_ϑ .

Deterministic decomposition and external product: Any $\mathbf{v} \in \hat{R}_q^{1 \times (d+1)}$ can be uniquely and deterministically decomposed into a small vector $\mathbf{G}^{-1}(\mathbf{v}) \in R^{1 \times (d+1) \cdot \ell}$ whose coefficients are integers in $[-B/2, B/2[$ and such that $\mathbf{G}^{-1}(\mathbf{v}) \cdot \mathbf{G} = \mathbf{v}$. The full details are provided in supplementary materials Section B.1. This allows to multiply a RGSW ciphertext $\mathbf{C}$ and a RLWE ciphertext $\mathbf{c}$:

$$\mathbf{C} \boxtimes \mathbf{c} = \mathbf{G}^{-1}(\mathbf{c}) \cdot \mathbf{C}$$

This external product allows homomorphic evaluations of multiplexers with convenient noise growth $\mathbf{c}_0 + \mathbf{C} \boxtimes (\mathbf{c}_1 - \mathbf{c}_0)$ for a RGSW encryption $\mathbf{C}$ of the selector bit, and RLWE encryptions of the two inputs $\mathbf{c}_0$ and $\mathbf{c}_1$.

TFHE Bootstrapping: The bootstrapping procedure is a common tool to all known FHE schemes which allows to manage the noise growth during homomorphic evaluations. More formally, the bootstrapping takes as input an LWE ciphertext of some message μ and a bootstrapping key BK and outputs an LWE ciphertext of the same message μ , whose noise is below some bound controlled via the choice of parameters. The TFHE bootstrapping procedure can be decomposed in four steps, that are illustrated in Figure 1. The four steps are described below, with their type signatures explicitly defined.

- Initialization: $n\text{-LWE} \rightarrow \mathbb{Z}_{2N}^{n+1} \times \hat{R}_q$.
Given an LWE ciphertext $(\mathbf{a} \mid b) \in \hat{\mathbb{Z}}_q^{n+1}$ as input, we define $\bar{b} = \lfloor 2Nb \rfloor$ and $\bar{a}_i = \lfloor 2Na_i \rfloor \in \mathbb{Z}_{2N}$ for each $i \in [n]$. Note that the cumulated error induced by rounding over $\mathbb{Z}_{2N}$ is taken into account in the definition of $\mathcal{C}_\mu$. The first step also initializes a test vector, $\text{testv} \stackrel{\text{def}}{=} v(X) \in \hat{R}_q$, $v(X) = \sum_{i=0}^{N-1} v_i X^i$ and defines a noiseless RLWE encryption $(\mathbf{0}, \text{testv})$. The test vector coefficients are chosen such that, after the BlindRotate, one can retrieve an LWE ciphertext of the input message as the constant coefficient of the RLWE ciphertext.
- BlindRotate: $\mathbb{Z}_{2N}^{n+1} \times \hat{R}_q \times \text{RGSW}^n \rightarrow \text{RLWE}$
Given $\text{testv} \in \hat{R}_q$, n coefficients $(\bar{a}_1, \dots, \bar{a}_n, \bar{b}) \in \mathbb{Z}_{2N}^{n+1}$, and a bootstrapping key $\text{BK}_i = \text{RGSW}_{\tilde{s}, \vartheta_{\text{BK}}}(s_i)$ for $i \in [n]$, the BlindRotate algorithm returns a RLWE encryption of $\text{testv} \cdot X^{\bar{\varphi}}$ where $\bar{\varphi} = \bar{b} - \sum_{i=1}^n s_i \bar{a}_i$.
- Extract: $\text{RLWE} \rightarrow dN\text{-LWE}$
By interpreting a RLWE ciphertext of message $\mu \in \hat{R}_q$ under $\tilde{s}$ as a vector of $d+1$ coefficients and taking into account the negacyclicity of the test vector polynomial i.e. $v_{i+N} = -v_i$, Extract extracts an LWE ciphertext under key $K = \text{KeyExtract}(\tilde{s})$ of dimension dN of the constant term of μ , $\mu(0)$.
- KeySwitch: $dN\text{-LWE} \times (n\text{-LWE})^{dNt} \rightarrow n\text{-LWE}$
Given a dN -LWE ciphertext containing a message μ under key K , and a keyswitching key which consists of n -LWE encryptions of the bits of key K multiplied by the first t powers of $\frac{1}{B_{\text{KS}}}$ under secret key $\mathbf{s} \in \mathbb{B}^n$, KeySwitch outputs an n -LWE ciphertext of the same message μ under secret key $\mathbf{s}$.

The security of TFHE thus relies on both the decision LWE problem as well as the decision RLWE problem, but also on a circular assumption that states having LWE and RLWE encryptions of each other's secret key has no impact on security.

3 Randomization over rings

In this section we show how to randomize RLWE ciphertexts, an essential building block for reaching the circuit privacy and the sanitization properties for FHE. These results are of independent interest and can be used directly in other works, including [DS16]. We present two solutions: the first one reaches the desired properties in a statistical sense, i.e. with security against unbounded adversaries. However, the parameters are unrealistic to expect an efficient implementation. The second one yields a very efficient randomization technique, but holds under the hardness of the decision RLWE problem.

3.1 Gaussian Lemma over rings

We start with the following lemma on discrete Gaussians, which is a generalization of the result of [BdPMW16] to the ring setting, to arbitrary lattices and to Gaussian distributions that may not be spherical. It will be used both for proving results over $\Lambda^\perp(\mathbf{G})$, and over $\frac{1}{q}R$.

Lemma 7. *Let $\varepsilon > 0$, $\Lambda_1 \subset R^{d_1}$ be a rank- $d_1 N$ lattice, $S_1 \in \mathbb{R}^{k_1 \times d_1 N}$ be a rank- $d_1 N$ matrix, $S_2 \in \mathbb{R}^{k_2 \times N}$ be a rank- N matrix. For any $\mathbf{e} = (e_1, \dots, e_{d_1}) \in \hat{R}_q^{d_1}$ and any $\mathbf{c} \in \hat{R}_q^{d_1}$, let $\mathbf{E}_i$ be the matrix such that $\text{pow}_X \cdot \mathbf{E}_i = \text{pow}_X \otimes e_i$ and $\mathbf{E} = (\mathbf{E}_0 \mid \mathbf{E}_1 \mid \dots \mid \mathbf{E}_{d_1})$, then*

if $\min(\sigma_{d_1 N}(S_1), \sigma_N(qS_2)) \geq (1 + q\sqrt{N} \|\mathbf{e}\|_2) \lambda_{d_1 N}(\Lambda) \cdot \sqrt{\frac{\ln(2d_1 N(1+1/\varepsilon))}{\pi}}$, we have

$$\Delta(\mathbf{e}^t \mathbf{x} + y, e') < 2\varepsilon$$

where $\mathbf{x} \leftarrow \mathcal{D}_{\Lambda_1 + \mathbf{c}, S_1}$, $y \leftarrow \mathcal{D}_{\frac{1}{q}R, S_2}$, and $e' \leftarrow \mathcal{D}_{\frac{1}{q}R, \Gamma}$, with

$$\Gamma = \sqrt{S_2^t S_2 + \mathbf{E} S_1^t S_1 \mathbf{E}^t}.$$

Proof. In order to be able to use known results on lattices, let us consider the naive coefficient embedding of the polynomials. That is, we consider $\mathbf{c} = (c_{1,0}, c_{1,1}, \dots, c_{1,N-1}, c_{2,0}, \dots, c_{d_1,N-1}) \in \frac{1}{q}\mathbb{Z}^{d_1 N}$. We define the following notations in order to better explain the intuition behind the proof:

- $\hat{\mathbf{E}} = \left(\mathbf{E} \mid \frac{1}{q}\mathbf{I}_N \right) \in \frac{1}{q}\mathbb{Z}^{N \times N(d_1+1)}$;
- $\beta = \begin{pmatrix} S_1 & \mathbf{0} \\ \mathbf{0} & qS_2 \end{pmatrix} \in \mathbb{R}^{k_1+k_2 \times N(d_1+1)}$;
- $\hat{\mathbf{c}} = (\mathbf{c}, 0, \dots, 0) \in \frac{1}{q}\mathbb{Z}^{N(d_1+1)}$;
- $\hat{\Lambda} = \Lambda_1 \times \mathbb{Z}^N \subset \mathbb{Z}^{N(d_1+1)}$.

We want to show that:

$$\Delta\left(\hat{\mathbf{E}} \mathcal{D}_{\hat{\Lambda} + \hat{\mathbf{c}}, \beta}, \mathcal{D}_{\frac{1}{q}\mathbb{Z}^N, \sqrt{\hat{\mathbf{E}} \beta^t \beta \hat{\mathbf{E}}^t}}\right) \leq 2\varepsilon$$

Both distributions have the same support: $\frac{1}{q}\mathbb{Z}^N$. Let us analyze the probability mass assigned to each element $\mathbf{z}$ of $\frac{1}{q}\mathbb{Z}^N$. We have

$$\left(\widehat{\mathbf{E}}\mathcal{D}_{\widehat{\Lambda}+\widehat{\mathbf{c}},\beta}\right)(\mathbf{z}) = \frac{\rho_\beta(\mathcal{L}_{\mathbf{z}})}{\sum \rho_\beta(\mathcal{L}_{\mathbf{z}'}), \quad \text{with } \mathcal{L}_{\mathbf{z}} = \left\{ \mathbf{v} \in \widehat{\Lambda} + \widehat{\mathbf{c}} \mid \widehat{\mathbf{E}}\mathbf{v} = \mathbf{z} \right\},$$

where the sum ranges over all cosets $\mathcal{L}_{\mathbf{z}'}$ of $\mathcal{L}_{\mathbf{0}}$, when $\mathbf{z}'$ ranges over $\frac{1}{q}\mathbb{Z}^N$. The idea is to show that $\rho_\beta(\mathcal{L}_{\mathbf{z}})$ is close to being proportional to $\rho_{\sqrt{\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t}}(\mathbf{z})$.

$$\rho_{\sqrt{\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t}}(\mathbf{z}) = \rho(\mathbf{u}_{\mathbf{z}}), \text{ with } \mathbf{u}_{\mathbf{z}} = \beta\widehat{\mathbf{E}}^t \left(\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t \right)^{-1} \mathbf{z} = \beta(\beta^t\beta)^{-1} \beta^t \mathbf{u}_{\mathbf{z}} \in \mathbb{R}^{k_1+k_2}.$$

For any $\mathbf{t} \in \mathcal{L}_{\mathbf{z}}$, $\rho_\beta(\mathbf{t}) = \rho(\mathbf{u}_{\mathbf{t}})$, with $\mathbf{u}_{\mathbf{t}} = \beta(\beta^t\beta)^{-1} \mathbf{t} \in \mathbb{R}^{k_1+k_2}$.

We have $\mathbf{u}_{\mathbf{z}}^t \mathbf{u}_{\mathbf{z}} = \mathbf{u}_{\mathbf{z}}^t \mathbf{u}_{\mathbf{z}}$, so $\mathbf{u}_{\mathbf{z}}$ is orthogonal to $\mathbf{u}_{\mathbf{t}} - \mathbf{u}_{\mathbf{z}}$, which gives the identity

$$\begin{aligned} \rho(\mathbf{u}_{\mathbf{t}}) &= \rho(\mathbf{u}_{\mathbf{z}}) \cdot \rho(\mathbf{u}_{\mathbf{t}} - \mathbf{u}_{\mathbf{z}}) \\ \rho_\beta(\mathbf{t}) &= \rho_{\sqrt{\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t}}(\mathbf{z}) \cdot \rho_\beta(\mathbf{t} - \beta^t \mathbf{u}_{\mathbf{z}}) \\ \rho_\beta(\mathcal{L}_{\mathbf{z}}) &= \rho_{\sqrt{\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t}}(\mathbf{z}) \cdot \rho_\beta(\mathcal{L}_{\mathbf{0}} + \mathbf{t} - \beta^t \mathbf{u}_{\mathbf{z}}) \end{aligned}$$

We have that $\mathcal{L}_{\mathbf{0}}$ is the sublattice of $\widehat{\Lambda}$ that is orthogonal to $\widehat{\mathbf{E}}$, with $\widehat{\Lambda}$ of dimension $N(d_1 + 1)$ and $\widehat{\mathbf{E}}$ of rank N , so $\mathcal{L}_{\mathbf{0}}$ is of dimension $d_1 N$.

Let $\mathbf{b}_1, \dots, \mathbf{b}_{d_1 N}$ be a set of $d_1 N$ independent vectors of Λ_1 of norm bounded by $\lambda_{d_1 N}(\Lambda)$. Then, $(\mathbf{b}_1, -q\mathbf{E}\mathbf{b}_1), \dots, (\mathbf{b}_{d_1 N}, -q\mathbf{E}\mathbf{b}_{d_1 N})$, is a set of $d_1 N$ independent vectors of $\mathcal{L}_{\mathbf{0}}$ of norm bounded by $(1 + q\sqrt{N}\|\mathbf{e}\|_2)\lambda_{d_1 N}(\Lambda)$. Using lemma 2 gives

$$\eta_\varepsilon(\mathcal{L}_{\mathbf{0}}) \leq (1 + q\sqrt{N}\|\mathbf{e}\|_2)\lambda_{d_1 N}(\Lambda_1) \cdot \sqrt{\frac{\ln(2d_1 N(1 + 1/\varepsilon))}{\pi}}.$$

And by lemma 4, $\rho_\beta(\mathcal{L}_{\mathbf{0}} + \mathbf{t} - \beta^t \mathbf{u}_{\mathbf{z}}) \in \left[\frac{1-\varepsilon}{1+\varepsilon}, 1 \right] \cdot \rho_\beta(\mathcal{L}_{\mathbf{0}})$.

Putting all back together, this implies that the statistical distance between $\widehat{\mathbf{E}}\mathcal{D}_{\widehat{\Lambda}+\widehat{\mathbf{c}},\beta}$ and $\mathcal{D}_{\frac{1}{q}\mathbb{Z}^N, \sqrt{\widehat{\mathbf{E}}\beta^t\beta\widehat{\mathbf{E}}^t}}$ is at most $1 - \frac{1-\varepsilon}{1+\varepsilon} \leq 2\varepsilon$.

3.2 Randomized decomposition

We now present the randomized decomposition which is one of the essential tools for randomizing RLWE ciphertexts. Since the gadget matrix $\mathbf{G}$ only has constant coefficients, the product of $\mathbf{G}$ with a vector of polynomials $\mathbf{u} \in R^{(d+1)\cdot\ell}$ operates independently on each entry of $\mathbf{u}$. We can thus extend the sampling function from Lemma 1 to vectors of polynomials.

Definition 3 (Randomized Decomposition). *We define the randomized decomposition function $\mathbf{G}_r^{-1}(\cdot) : \hat{R}_q^{1 \times (d+1)} \rightarrow R^{1 \times (d+1) \cdot \ell}$ by using N copies of the randomized decomposition over $\hat{\mathbb{Z}}_q^{d+1}$ given by Lemma 1 with parameter r , i.e. we decompose each coefficient independently.*

We also define the randomized external product $\boxtimes_r$, which is defined as the external product $\boxtimes$ from Section 2.6, replacing $\mathbf{G}^{-1}(\cdot)$ by the randomized decomposition $\mathbf{G}_r^{-1}(\cdot)$.

In the following, we denote $\vartheta_{\square}$ the variance of a Gaussian of parameter r . We have $\vartheta_{\square} = \frac{r^2}{2\pi q^2}$. We give the full details of the randomized decomposition in the supplementary materials Section B.2. The following lemma, which is an adaptation of [CGGI20, Corollary 3.14] bounds the noise variance of the randomized external product.

Lemma 8 (Variance of randomized External Product). *For any $\mu \in \hat{R}_q$, ciphertext $\mathbf{c} \in \text{RLWE}_{\tilde{s}}(\mu)$ and $\mathbf{C} = \text{RGSW}_{\tilde{s}, \vartheta_{\mathbf{C}}}(0)$ a fresh encryption of 0. Then $\mathbf{C} \boxdot_r \mathbf{c} \in \text{RLWE}_{\tilde{s}}(0)$ and*

$$\|\text{Var}(\mathbf{C} \boxdot_r \mathbf{c})\|_2 \leq 2\pi q^2 \vartheta_{\square} (d+1) \ell N \vartheta_{\mathbf{C}}$$

except with probability at most $2^{-2(d+1) \cdot \ell \cdot N}$.

Proof. Let $\mathbf{C} = (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e})$, with $\mathbf{A} \in \hat{R}_q^{(d+1) \cdot \ell \times d}$ and $\mathbf{e} \in \hat{R}_q^{(d+1) \cdot \ell}$. We have:

$$\begin{aligned} \mathbf{C} \boxdot_r \mathbf{c} &= \mathbf{G}_r^{-1}(\mathbf{c}) \cdot \mathbf{C} \\ &= (\mathbf{G}_r^{-1}(\mathbf{c}) \cdot \mathbf{A} \mid \mathbf{G}_r^{-1}(\mathbf{c}) \cdot \mathbf{A}\tilde{\mathbf{s}} + \mathbf{G}_r^{-1}(\mathbf{c}) \cdot \mathbf{e}) \end{aligned}$$

So $\text{Err}(\mathbf{C} \boxdot_r \mathbf{c}) = \mathbf{G}_r^{-1}(\mathbf{c}) \cdot \mathbf{e}$. Using Lemma 3, the euclidean norm of $\mathbf{G}_r^{-1}(\mathbf{c})$ is bounded by $r \cdot \sqrt{(d+1) \cdot \ell \cdot N}$ except with probability at most $2^{-2(d+1) \cdot \ell \cdot N}$. Since all coefficients of $\mathbf{e}$ are independent, we conclude using the additive property of covariance matrices.

The following corollary instantiates Lemma 7 with the notations of our randomized decomposition.

Corollary 1. *For any $\mathbf{e} = (e_1, \dots, e_{(d+1) \cdot \ell}) \in \hat{R}_q^{(d+1) \cdot \ell}$ and any $\mathbf{c} \in \hat{R}_q^{(d+1) \cdot \ell}$, if $\min(\vartheta_{\square}, \vartheta_y) \geq (\frac{1}{q} + \sqrt{N} \|\mathbf{e}\|_2)^2 (1 + B^2) \cdot \frac{\ln(2 \cdot (d+1) \cdot \ell \cdot N \cdot (1+1/\varepsilon))}{2\pi^2}$, we have:*

$$\Delta \left(\mathbf{e}^t \mathcal{D}_{\Lambda^\perp(\mathbf{G}) + \mathbf{c}, q\sqrt{2\pi\vartheta_{\square}}} + \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_y}}, \mathcal{D}_{\frac{1}{q}R, \Gamma} \right) \leq 2\varepsilon,$$

$$\text{with } \Gamma = \sqrt{\frac{\vartheta_y}{2\pi} \mathbf{I}_N + \frac{\vartheta_{\square}}{2\pi} q^2 \sum_{i=1}^{(d+1) \cdot \ell} \mathbf{E}_i^t \mathbf{E}_i}.$$

3.3 Leftover Hash Lemma over rings

The following Lemma is a variant of the Leftover Hash Lemma adapted to $\hat{R}_q$ where q is a power of two. Generalized versions of the Leftover Hash Lemma or regularity lemma over rings already exist in the literature (e.g., [MM11, LPR13, LW20, DGKS21]), but some of these results do not apply to our case, either because q is not prime in our case, either because we are interested in lattice cosets. Also, by using the generalized result from [LW20], we would still need to compute the probability of collision over the ideals of $\hat{R}_q$, which is the main technical difficulty of our proof. We give a detailed proof in Appendix C using the general result from [MM11], but we chose to prove it from scratch for the sake of completeness. Lemma 9 shows that the leakage of $\mathbf{e}^t \mathbf{x} + \mathbf{y}$ is negligible and that we can still apply a variant of the Leftover Hash Lemma as long as the dimension ℓ is sufficiently large and $\mathbf{x}$ has sufficient entropy.

Lemma 9. *Let $\epsilon_\ell, \epsilon, r > 0$. Under the following conditions:*

$$\begin{aligned} - \ell &> \frac{1}{d+1} \left(2d - 1 + \log \left(\frac{1-2^{dNk}}{1-2^d} \right) + \log \left(\frac{1+\epsilon}{1-\epsilon} \right) + Nk - 2 \log(\epsilon_\ell) \right), \\ - r &\geq 2\sqrt{(1+B^2) \cdot \frac{\ln(2N(d+1) \cdot \ell(1+1/\epsilon))}{\pi}}, \end{aligned}$$

for any $\mathbf{e} \in \hat{R}_q^{(d+1)\cdot\ell}$, $\mathbf{y} \in \hat{R}_q$, $\mathbf{c} \in \hat{R}_q^{(d+1)\cdot\ell}$:

$$\Delta\left((\mathbf{A}, \mathbf{x}^t \cdot \mathbf{A}, \mathbf{e}^t \mathbf{x} + \mathbf{y}), (\mathbf{A}, \mathbf{u}^t, \mathbf{e}^t \mathbf{x} + \mathbf{y})\right) < \varepsilon_\ell$$

where $\mathbf{A}$ is uniform over $\hat{R}_q^{(d+1)\cdot\ell \times d}$, $\mathbf{x} \leftarrow \mathcal{D}_{\Lambda^\perp(\mathbf{G})+\mathbf{c}, r}$, $\mathbf{u}$ is uniform over $\hat{R}_q^d$.

Using Lemma 9 and Lemma 7, Section 3.5 shows that combining the randomized decomposition with an additional small Gaussian noise is enough to randomize RLWE ciphertexts as long as the parameters are large enough.

3.4 Using a sanitization key

In case the bounds required by Lemma 9 cannot be met, we propose an alternative randomization process which makes use of an additional public key encryption of zero. This technique enables to reach more efficient parameters and allows us to devise a practical schemes, for which the randomization guarantees rely on the hardness of the decision RLWE problem.

We note that appending this sanitization key PK to the evaluation key gives more power to an attacker. However, the semantic security still relies on the decision variant of LWE over rings, together with the circular security assumption. We just assume more RLWE samples at hand.

In order for our result to hold, we require the following properties on the public encryption scheme to be used, and we provide a construction that satisfies those conditions. Intuitively, we require that the ciphertext looks uniform to an adversary knowing the secret key, but without having knowledge of the noise and the message.

Lemma 10 (Sanitization key). *There exist two algorithms PkGen and PkEnc such that:*

$\text{PkGen}(1^\lambda, \tilde{\mathbf{s}})$: on input a security parameter λ , and a RLWE secret key $\tilde{\mathbf{s}}$, PkGen outputs a public key PK.

$\text{PkEnc}(\text{PK})$: for any RLWE secret key $\tilde{\mathbf{s}}$, the output of PkEnc satisfies:

$$(\text{PK}, \text{PkEnc}(\text{PK})) \approx_c (\text{PK}, (\mathbf{u} \mid \mathbf{u}\tilde{\mathbf{s}} + e_u))$$

where $\mathbf{u}$ is uniform over R_q^d , e_u has a variance bounded by some value ϑ_{PK} with overwhelming probability and PK is honestly generated from $\text{PkGen}(1^\lambda, \tilde{\mathbf{s}})$.

The distribution of e_u is not important for the sanitization property but it has to remain small for correctness.

One possible solution that reaches the statistical closeness would be to use a public key a la Regev, with PK being a set of $\omega((d+1)N \log(q))$ encryptions of 0, and $\text{PkEnc}(\text{PK})$ being the sum of a random subset of those. However, the size of the public key and the computational complexity to generate it will be prohibitive in practice. Instead, we leverage the RLWE decisional assumption to reduce the size of PK and the time to generate $\text{PkEnc}(\text{PK})$ to almost negligible. We believe that the analysis of the distribution of ciphertexts produced by $\text{PkEnc}(\text{PK})$, its generalization to higher dimension and its application are of independent interest. For example, the public key can be used as a randomizer in combination with the soak-and-spin technique of [DS16] to obtain better key size, efficiency and noise management trade-offs. More details are given in Section A.

Construction 1 *We exhibit a concrete example of a construction satisfying the property of Lemma 10.*

- $\text{PkGen}(1^\lambda, \tilde{\mathbf{s}})$: on input the security parameter λ , and a secret key $\tilde{\mathbf{s}} \in R^d$,

$$\text{PK} = (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) \in \hat{R}_q^{d' \times d+1},$$

where $\mathbf{A}$ is uniform over $\hat{R}_q^{d' \times d}$, and each coefficient of $\mathbf{e} \in \hat{R}_q^{d'}$ has coefficients sampled from $\chi_{\vartheta_{\mathbf{e}}}$, for a noise variance $\vartheta_{\mathbf{e}}$.

- $\text{PkEnc}(\text{PK})$: on input public key PK , returns

$$\mathbf{r} \cdot \text{PK} + (\mathbf{e}' \mid e'') \in \hat{R}_q^{1 \times d+1},$$

where $\mathbf{r} \leftarrow \mathcal{D}_{R^{1 \times d'}, q\sqrt{2\pi\vartheta_{\mathbf{r}}}}$, $\mathbf{e}' \leftarrow \mathcal{D}_{\frac{1}{q}R^{1 \times d}, \sqrt{2\pi\vartheta_{\mathbf{e}'}}}$ and $e'' \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_{e''}}}$

Lemma 11. Under the hardness assumption of the RLWE decision problem with noise variance ϑ , where $\frac{1}{\vartheta} = 2 \left(\min \left(\frac{1}{\vartheta_{\mathbf{r}}}, \frac{1}{\vartheta_{\mathbf{e}'}} \right) + \frac{N}{\vartheta_{e''}} \left(\|\mathbf{q}\mathbf{e}\|_2^2 + \|\tilde{\mathbf{s}}\|_2^2 \right) \right)$ as long as the following condition is met:

$$\vartheta \geq \frac{1}{\pi} \eta_{\varepsilon} \left(\frac{1}{q} \mathbb{Z}^{(d'+d)N} \right)^2$$

Construction 1 satisfies the property of Lemma 10, with

$$\vartheta_{\text{PK}} = \vartheta_{e''} + q^2 \vartheta_{\mathbf{r}} \|\mathbf{e}\|_2^2 + \|\tilde{\mathbf{s}}\|_2^2 \vartheta_{\mathbf{e}'},$$

To prove Lemma 11, we will use the following result, which is an adaptation of [KLSS23, Lemma 7] to our notations. We provide a detailed proof in Appendix D for the sake of completeness.

Lemma 12. For any $\tilde{\mathbf{s}} \in R^d, \mathbf{e} \in \hat{R}_q^{d'}$, let $\tilde{\mathbf{S}}_i$ be the matrix such that $\text{pow}_X \cdot \tilde{\mathbf{S}}_i = \text{pow}_X \otimes \tilde{s}_i$, and $\mathbf{E}_i$ be the matrix such that $\text{pow}_X \cdot \mathbf{E}_i = \text{pow}_X \otimes e_i$. We define $\Gamma = \left(q\mathbf{E}_1^t \mid \dots \mid q\mathbf{E}_{d'}^t \mid -\tilde{\mathbf{S}}_1^t \mid \dots \mid -\tilde{\mathbf{S}}_d^t \right)$. Note that $\text{pow}_X \cdot \Gamma = \text{pow}_X \otimes (q\mathbf{e}^t \mid -\tilde{\mathbf{s}}^t)$. We note

$$\Sigma = \begin{pmatrix} 2\pi\vartheta_{\mathbf{r}}\mathbf{I}_{d'N} & \mathbf{0} \\ \mathbf{0} & 2\pi\vartheta_{\mathbf{e}'}\mathbf{I}_{dN} \end{pmatrix}, \quad \Sigma_{\mathbf{v}} = \left(\Sigma^{-1} + \frac{1}{2\pi\vartheta_{e''}} \Gamma^t \Gamma \right)^{-1}.$$

$$\Delta \left((\mathbf{v}, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e''), (\mathbf{v}_0 + \mathbf{v}^*, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'') \right) = 0,$$

where $\mathbf{r} \leftarrow \mathcal{D}_{R^{1 \times d'}, q\sqrt{2\pi\vartheta_{\mathbf{r}}}}$, $\mathbf{e}' \leftarrow \mathcal{D}_{\frac{1}{q}R^{1 \times d}, \sqrt{2\pi\vartheta_{\mathbf{e}'}}}$, $e'' \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_{e''}}}$, $\mathbf{v} = \left(\frac{1}{q} \mathbf{r}^t, \mathbf{v}_0 = \frac{1}{2\pi\vartheta_{e''}} \Sigma_{\mathbf{v}} \begin{pmatrix} q\mathbf{e} \\ -\tilde{\mathbf{s}} \end{pmatrix} (\mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'') \right)$ and $\mathbf{v}^* \leftarrow \mathcal{D}_{-\mathbf{v}_0 + \frac{1}{q}R^{d'+d}, \sqrt{\Sigma_{\mathbf{v}}}}$.

We are now ready to prove Lemma 11:

Proof. For any secret key $\tilde{\mathbf{s}} \in R^d$, and any $\mathbf{e} \in \hat{R}_q^{d'}$, let $\tilde{\mathbf{S}}_i$ (resp. $\mathbf{E}_i$) be the matrix such that $\text{pow}_X \cdot \tilde{\mathbf{S}}_i = \text{pow}_X \otimes \tilde{s}_i$ (resp. $\text{pow}_X \cdot \mathbf{E}_i = \text{pow}_X \otimes e_i$). We set

$$\Gamma = \left(q\mathbf{E}_1^t \mid \dots \mid q\mathbf{E}_{d'}^t \mid -\tilde{\mathbf{S}}_1^t \mid \dots \mid -\tilde{\mathbf{S}}_d^t \right), \quad \Sigma_{\mathbf{v}}^{-1} = \frac{1}{2\pi\vartheta_{e''}} \Gamma^t \Gamma + \frac{1}{2\pi} \begin{pmatrix} \vartheta_{\mathbf{r}}\mathbf{I}_{d'N} & \mathbf{0} \\ \mathbf{0} & \vartheta_{\mathbf{e}'}\mathbf{I}_{dN} \end{pmatrix}^{-1}.$$

Lemma 12 gives us:

$$(\mathbf{r}, \mathbf{e}', \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'') = (\mathbf{r}_0 + \mathbf{r}^*, \mathbf{e}'_0 + \mathbf{e}'^*, e_u), \quad (2)$$

with $e_u = \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e''$,

$$\mathbf{v}_0 = \begin{pmatrix} \frac{1}{q}\mathbf{r}_0^t \\ \mathbf{e}_0'^t \end{pmatrix} = \frac{1}{2\pi\vartheta_{e''}} \Sigma_{\mathbf{v}} \begin{pmatrix} q\mathbf{e} \\ -\tilde{\mathbf{s}} \end{pmatrix} e_u, \quad \mathbf{v}^* \leftarrow \mathcal{D}_{-\mathbf{v}_0 + \frac{1}{q}R^{d'+d}, \sqrt{\Sigma_{\mathbf{v}}}}$$

Next, we apply lemma 5 and $\mathbf{v}^*$ can be split into $\mathbf{v}^\dagger + \hat{\mathbf{v}}$, with $\mathbf{v}^\dagger \leftarrow \mathcal{D}_{\frac{1}{q}R^{d'+d}, \sqrt{2\pi\vartheta}}$ and $\hat{\mathbf{v}} \leftarrow \mathcal{D}_{-\mathbf{v}_0 + \frac{1}{q}R^{d'+d}, \sqrt{\hat{\Sigma}}}$, with $\hat{\Sigma} = \Sigma_{\mathbf{v}} - 2\pi\vartheta\mathbf{I}_{(d'+d)N}$, by Lemma 5. With this choice, the conditions of Lemma 5 indeed hold since : $\hat{\Sigma}$ is positive definite (its smallest singular value is strictly positive), we have $\sigma_{(d'+d)N}(\Sigma_{\mathbf{v}})$ is at least:

$$\left(\max \left(\frac{1}{2\pi\vartheta_{\mathbf{r}}}, \frac{1}{2\pi\vartheta_{\mathbf{e}'}} \right) + \frac{N}{2\pi\vartheta_{e''}} \left(\|q\mathbf{e}\|_2^2 + \|\tilde{\mathbf{s}}\|_2^2 \right) \right)^{-1} \geq 4\pi\vartheta > 2\pi\vartheta, \text{ and}$$

$$\left\| (2\pi\vartheta\mathbf{I}_{(d'+d)N})^{-1} + \hat{\Sigma}^{-1} \right\|_2 = \frac{1}{2\pi\vartheta} + \sigma_{(d'+d)N}(\hat{\Sigma})^{-1} \leq \frac{1}{\pi\vartheta} \leq \eta_\varepsilon \left(\frac{1}{q}\mathbb{Z}^{(d'+d)N} \right)^{-2}$$

which implies by lemma 6 that:

$$\sigma_{(d'+d)N} \left(\left((2\pi\vartheta\mathbf{I}_{(d'+d)N})^{-1} + \hat{\Sigma}^{-1} \right)^{-1} \right) \geq \eta_\varepsilon \left(\frac{1}{q}\mathbb{Z}^{(d'+d)N} \right).$$

We can then conclude by invoking the hardness of the decision RLWE problem with secret $\mathbf{r}^\dagger$. To sum up, we have the following hybrid argument

$$\begin{aligned} \text{PK}, \text{PkEnc}(\text{PK}) &= \text{PK}, (\mathbf{r}\mathbf{A} + \mathbf{e}' \mid (\mathbf{r}(\mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) + e'')) \\ &= \text{PK}, (\mathbf{r}\mathbf{A} + \mathbf{e}' \mid \mathbf{r}\mathbf{A}\tilde{\mathbf{s}} + \mathbf{r}\mathbf{e} + e'') \\ &= \text{PK}, \left((\mathbf{r}_0 + \mathbf{r}^*)\mathbf{A} + \mathbf{e}'_0 + \mathbf{e}'^* \mid \right. && \text{(by Lemma 12)} \\ &\quad \left. ((\mathbf{r}_0 + \mathbf{r}^*)\mathbf{A} + \mathbf{e}'_0 + \mathbf{e}'^*)\tilde{\mathbf{s}} + e_u \right) \\ &\approx_s \text{PK}, \left((\mathbf{r}_0 + \hat{\mathbf{r}})\mathbf{A} + \mathbf{e}'_0 + \hat{\mathbf{e}}' + \mathbf{r}^\dagger\mathbf{A} + \mathbf{e}'^\dagger \mid \right. && \text{(by Lemma 5)} \\ &\quad \left. ((\mathbf{r}_0 + \hat{\mathbf{r}})\mathbf{A} + \mathbf{e}'_0 + \hat{\mathbf{e}}' + \mathbf{r}^\dagger\mathbf{A} + \mathbf{e}'^\dagger)\tilde{\mathbf{s}} + e_u \right) \\ &\approx_c \text{PK}, \left((\mathbf{r}_0 + \hat{\mathbf{r}})\mathbf{A} + \mathbf{e}'_0 + \hat{\mathbf{e}}' + \mathbf{u} \mid \right. && \text{(by RLWE)} \\ &\quad \left. ((\mathbf{r}_0 + \hat{\mathbf{r}})\mathbf{A} + \mathbf{e}'_0 + \hat{\mathbf{e}}' + \mathbf{u})\tilde{\mathbf{s}} + e_u \right) \\ &= \text{PK}, (\mathbf{u} \mid \mathbf{u}\tilde{\mathbf{s}} + e_u), \end{aligned}$$

The bound on ϑ_{PK} is obtained by bounding the variance of $\mathbf{r}\mathbf{e} + e'' - \mathbf{e}'\tilde{\mathbf{s}}$, using Lemma 3 to bound $\|\mathbf{r}\|_2$. $\square$

3.5 Randomization of RLWE ciphertexts

Our procedure to scale and randomize a RLWE ciphertext consists of a randomized decomposition, and the addition of a randomizer RERAND. In order to clearly expose the two strategies, we define:

- RERAND = $(\mathbf{0}, y)$ and $\approx$ denotes $\approx_s$, if $\ell > \frac{1}{d+1} \left(\log \frac{(Nk-1)(1+\varepsilon)}{4\epsilon_\ell^2(1-\varepsilon)} + d \right)$;
- RERAND = $(\mathbf{0}, y) + \text{PkEnc}(\text{PK})$ and $\approx$ denotes $\approx_c$ otherwise,

with $y \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_\square}}$ and PK honestly generated from $\text{PkGen}(1^\lambda, \tilde{\mathbf{s}})$.

The following lemma states that this randomization technique brings any encryption of zero close to a canonical distribution.

Lemma 13. For any $\tilde{\mathbf{s}} \in \hat{R}_q^d$, any $\mathbf{e} = (e_1, \dots, e_{(d+1) \cdot \ell}) \in \hat{R}_q^{(d+1) \cdot \ell}$, and any $\mathbf{v} \in \hat{R}_q^{d+1}$, we have:

$$\text{RERAND} + (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) \square_r \mathbf{v} \approx \text{RERAND} + (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) \square_r \mathbf{0},$$

where $\mathbf{A}$ is uniform over $\hat{R}_q^{(d+1) \cdot \ell \times d}$.

Proof. Assuming $\text{RERAND} = (\mathbf{0}, y)$, with $y \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_\square}}$, we have:

$$\begin{aligned} \text{RERAND} + (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) \square_r \mathbf{v} &= (\mathbf{A} \square_r \mathbf{v} \mid (\mathbf{A} \square_r \mathbf{v})\tilde{\mathbf{s}} + \mathbf{e} \square_r \mathbf{v} + y) \\ &\approx_s (\mathbf{u} \mid \mathbf{u} \cdot \tilde{\mathbf{s}} + \mathbf{e} \square_r \mathbf{v} + y) \text{ by Lemma 9} \\ &\approx_s (\mathbf{u} \mid \mathbf{u} \cdot \tilde{\mathbf{s}} + e') \text{ by Corollary 1,} \end{aligned}$$

where $\mathbf{u}$ is uniform over $\hat{R}_q^{1 \times d}$ and $e' \leftarrow \mathcal{D}_{\frac{1}{q}R, \Gamma}$ for some Γ given by Corollary 1.

Assuming $\text{RERAND} = (\mathbf{0}, y) + \text{PkEnc}(\text{PK})$, with $y \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_\square}}$ and PK honestly generated from $\text{PkGen}(1^\lambda, \tilde{\mathbf{s}})$, by Lemma 10, we have:

$$\begin{aligned} \text{RERAND} + (\mathbf{A} \mid \mathbf{A}\tilde{\mathbf{s}} + \mathbf{e}) \square_r \mathbf{v} &\approx_c (\mathbf{u} + \mathbf{A} \square_r \mathbf{v} \mid (\mathbf{u} + \mathbf{A} \square_r \mathbf{v})\tilde{\mathbf{s}} + \mathbf{e} \square_r \mathbf{v} + y + e_u) \\ &= (\mathbf{u} \mid \mathbf{u}\tilde{\mathbf{s}} + \mathbf{e} \square_r \mathbf{v} + y + e_u) \\ &\approx_s (\mathbf{u} \mid \mathbf{u}\tilde{\mathbf{s}} + e' + e_u) \text{ by Corollary 1,} \end{aligned}$$

where $\mathbf{u}$ is uniform over $\hat{R}_q^{1 \times d}$, the bound on the variance of e_u is given by Lemma 11, and $e' \leftarrow \mathcal{D}_{\frac{1}{q}R, \Gamma}$ for some Γ given by Corollary 1. □

The statement of Lemma 13 holds for any secret key $\tilde{\mathbf{s}}$, but as we pointed it out in Remark 1 of the preliminary section, the randomness in PkGen and A is required to be honestly generated by the adversary.

4 New sanitization algorithm

In this section, we first present our circuit private version of the **BlindRotate** algorithm, which will be used as a building block for our sanitization procedure. We then introduce the new **Sanitize** procedure and show that it verifies the desired properties.

4.1 Circuit Private Blind Rotation

In order to achieve circuit privacy for the family of **BlindRotate** algorithms with hardwired inputs $(\bar{\mathbf{a}}, \bar{\mathbf{b}})$, the main loop of TFHE **BlindRotate** (line 3 of Algorithm 1, which is a multiplexer operation) is replaced by its randomized counterpart. We show by induction that adding one fresh randomizer at the end of the computation is enough to hide all information about $(\bar{\mathbf{a}}, \bar{\mathbf{b}})$ except for the result⁴. As opposed to the proof of circuit privacy from [BdPMW16], we start our hybrids by replacing *the last steps* of computation with an ideal world simulation where the accumulator holds value $\mathbf{0}$. Doing so in this reverse order shows that adding the randomizer only once is enough to prove that the computation is circuit private, while the proof technique of [BdPMW16] requires adding a randomizer at each step of computation. Hence, plugging our new technique in their work yields an improvement in the parameters.

⁴ The proof by induction is very crucial to ensure independence between variables and ensure Lemma 13 can be applied, since it requires the Gaussian samples to be fresh. This key point was missed in [Klu22] since they replace all the Gaussian samples at once, even though the distribution of each of them is defined by the previous ones.

Algorithm 1 Private computation of a RLWE encryption of $\text{testv} \cdot X^{\bar{\varphi}}$ where $(\bar{\mathbf{a}}, \bar{b}) \in \mathbb{Z}_{2N}^{n+1}$, and $\bar{\varphi} = \bar{b} - \sum_{i=1}^n s_i \bar{a}_i$

Input: $(\bar{\mathbf{a}}, \bar{b}) \in \mathbb{Z}_{2N}^{n+1}$, a bootstrapping key $\text{BK} = (\text{BK}_i)_i$, where BK_i is a RGSW encryption of s_i for $i \in [1, n]$, a fresh randomizer RERAND .

Output: $\text{CP-BlindRotate}_{\text{testv}}((\bar{\mathbf{a}}, \bar{b}), \text{BK}, \text{PK})$: a RLWE encryption of $\text{testv} \cdot X^{\bar{\varphi}}$ where $\bar{\varphi} = \bar{b} - \sum_{i=1}^n s_i \bar{a}_i$ and whose distribution is statistically close to a distribution independent from $(\bar{\mathbf{a}}, \bar{b})$, except for the message part.

- 1: $\text{ACC} = (0, \dots, 0, \text{testv} \cdot X^{-\bar{b}}) \in \hat{R}_q^{d+1}$,
- 2: **for** $i = 1$ **to** $n - 1$
- 3: $\text{ACC} += \text{BK}_i \boxtimes_r ((X^{\bar{a}_i} - 1) \text{ACC})$
- 4: $\text{ACC} += \text{BK}_n \boxtimes_r ((X^{\bar{a}_n} - 1) \text{ACC}) + \text{RERAND}$
- 5: **Return** ACC

Lemma 14. For any $\text{testv} \in \hat{R}_q$, the encryption scheme of Section 2.6 is circuit private for the class of functions $(\text{CP-BlindRotate}_{\text{testv}}((\bar{\mathbf{a}}, \bar{b}), \cdot, \text{RERAND}))_{(\bar{\mathbf{a}}, \bar{b})}$, if RERAND is a fresh randomizer as defined in Section 3.5.

Proof. For $0 \leq t \leq n$, let us denote ACC_t the value of the accumulator ACC after t iterations of the loop, and ACC^* the value of ACC after adding the randomizer RERAND . For $1 \leq t \leq n$, after the t -th iteration we have:

$$\begin{aligned} \text{ACC}_t &= \text{BK}_t \boxtimes_r ((X^{\bar{a}_t} - 1) \text{ACC}_{t-1}) + \text{ACC}_{t-1} \\ &= (\text{BK}_t - s_t \mathbf{G} + s_t \mathbf{G}) \boxtimes_r ((X^{\bar{a}_t} - 1) \text{ACC}_{t-1}) + \text{ACC}_{t-1} \\ &= (\text{BK}_t - s_t \mathbf{G}) \boxtimes_r ((X^{\bar{a}_t} - 1) \text{ACC}_{t-1}) + (1 + (X^{\bar{a}_t} - 1) s_t) \text{ACC}_{t-1} \end{aligned}$$

Since $1 + (X^{\bar{a}_t} - 1) s_t = X^{\bar{a}_t s_t}$, we have,

$$\text{ACC}_t = X^{\bar{a}_t s_t} \text{ACC}_{t-1} + (\text{BK}_t - s_t \mathbf{G}) \boxtimes_r ((X^{\bar{a}_t} - 1) \text{ACC}_{t-1}) \quad (3)$$

For the sake of readability, we define the following shorthand notations for given $\bar{\mathbf{a}}$ and $\mathbf{s}$: $X^{\geq t}$ denotes X raised to the power $\sum_{i=t}^n s_i \bar{a}_i$. We prove by induction that for t from n down to 0, the following holds:

$$\text{ACC}^* \approx X^{\geq t+1} \text{ACC}_t + \sum_{j=t+1}^n (\text{BK}_j - s_j \mathbf{G}) \boxtimes_r \mathbf{0} + \text{RERAND} \quad (4)$$

For $t = n$, we have $\text{ACC}^* = \text{ACC}_n + \text{RERAND}$.

Now assume that (4) holds at step $1 \leq t \leq n$, we show that the statement still holds at step $t - 1$. Using (3), we have:

$$\begin{aligned} \text{ACC}^* &\approx X^{\geq t+1} \text{ACC}_t + \sum_{j=t+1}^n (\text{BK}_j - s_j \mathbf{G}) \boxtimes_r \mathbf{0} + \text{RERAND} \\ &= X^{\geq t+1} (X^{\bar{a}_t s_t} \text{ACC}_{t-1} + (\text{BK}_t - s_t \mathbf{G}) \boxtimes_r ((X^{\bar{a}_t} - 1) \text{ACC}_{t-1})) \\ &\quad + \sum_{j=t+1}^n (\text{BK}_j - s_j \mathbf{G}) \boxtimes_r \mathbf{0} + \text{RERAND} \end{aligned}$$

For any $\mathbf{v} \in \hat{R}_q^{d+1}$, $X^i \Lambda^\perp(\mathbf{G}) = \Lambda^\perp(\mathbf{G})$, and $\|X^{-i} \cdot \mathbf{v}\|_2 = \|\mathbf{v}\|_2$, because they have the same coefficients modulo $X^{2N} - 1$ in absolute value. So for any parameter $r \in \mathbb{R}$, $X^i \mathbf{G}_r^{-1}(\mathbf{v}) = \mathbf{G}_r^{-1}(X^i \cdot \mathbf{v})$. This implies that

$$X^{\geq t+1} ((\text{BK}_t - s_t \mathbf{G}) \square_r \mathbf{v}) = (\text{BK}_t - s_t \mathbf{G}) \square_r (X^{\geq t+1} \cdot \mathbf{v})$$

We conclude using Lemma 13.

Finally, when $t = 0$, $X^{-\bar{b}} \cdot X^{\geq 0} = X^{\bar{\varphi}}$ and we have:

$$\text{ACC}^* = (\mathbf{0}, \text{testv} \cdot X^{\bar{\varphi}}) + \sum_{j=1}^n (\text{BK}_j - s_j \mathbf{G}) \square_r \mathbf{0} + \text{RERAND},$$

which is independent of $\bar{a}$ and $\bar{b}$, except for the result $\text{testv} \cdot X^{\bar{\varphi}}$. The following simulator $\text{Sim}_{cp}(\text{testv} \cdot X^{\bar{\varphi}}, \text{BK}, \text{RERAND})$ thus correctly simulates the output of $\text{CP-BlindRotate}_{\text{testv}}((\bar{\mathbf{a}}, \bar{\mathbf{b}}), \text{BK}, \text{RERAND})$:

$$\text{CP-BlindRotate}_{\text{testv}}((\mathbf{0}, \mathbf{0}), \text{BK}, \text{RERAND}) + (\mathbf{0}, \text{testv} \cdot (X^{\bar{\varphi}} - 1))$$

In order to analyze the correctness of our scheme, we make use of the fact that we described a distribution ACC^* that is indistinguishable from the real output of CP-BlindRotate that is much nicer to analyze. In fact ACC^* is a sum of independent Gaussian random variables which allows us to prove the correctness of the circuit private blind rotation (Lemma 15) without relying on the Gaussian Heuristic as in the TFHE framework. With this simplification, correctness can only be impacted by a negligible amount.

Lemma 15 (CP-BlindRotate noise propagation). *Let ACC^* be defined as in the proof of Lemma 14. We have that:*

$$\|\text{Var}(\text{ACC}^*)\|_2 \leq n2\pi q^2 \vartheta_{\square} (d+1) \ell N \vartheta_{\text{BK}} + \vartheta_{\text{RERAND}}$$

where ϑ_{BK} is the variance of the noise in the bootstrapping key BK , $\vartheta_{\square}$ is the variance of the Gaussians samples in $\square_r$, and

- $\vartheta_{\text{RERAND}} = \vartheta_{\square}$ if $\text{RERAND} = (\mathbf{0}, y)$;
- $\vartheta_{\text{RERAND}} = \vartheta_{\square} + \vartheta_{\text{PK}}$ if $\text{RERAND} = (\mathbf{0}, y) + \text{PkEnc}(\text{PK})$,

with ϑ_{PK} the variance of $\text{PkEnc}(\text{PK})$.

Proof. As $\text{testv} \cdot X^{\bar{\varphi}}$ is a constant random variable and that RERAND is made of the sum of a fresh Gaussian error and a fresh encryption of zero, we have:

$$\|\text{Var}(\text{ACC}^*)\|_2 \leq \left\| \text{Var} \left(\sum_{j=1}^n \mathbf{G}_r^{-1}(\mathbf{0}) \cdot (\text{BK}_j - s_j \mathbf{G}) \right) \right\|_2 + \vartheta_{\text{PK}} + \vartheta_{\square}$$

Since the n samples $\mathbf{G}_r^{-1}(\mathbf{0}) \cdot (\text{BK}_j - s_j \mathbf{G})$ are independent, summing n times the bound from Lemma 8 gives:

$$\|\text{Var}(\text{ACC}^*)\|_2 \leq n2\pi q^2 \vartheta_{\square} (d+1) \ell N \vartheta_{\text{BK}} + \vartheta_{\text{PK}} + \vartheta_{\square}$$

Algorithm 2 Sanitizing algorithm for LWE encryption $(\mathbf{a}, b)$ of μ

Input: $(\mathbf{a}, b)$ an LWE encryption of μ , a bootstrapping key $\text{BK} = (\text{BK}_i)_i$, where BK_i is a RGSW encryption of s_i for $i \in [1, n]$, a fresh randomizer RERAND , a keyswitching key KS , and two fixed messages $\mu_0 (= 0), \mu_1 (= \frac{1}{2})$.

Output: an LWE encryption $(\mathbf{a}', b')$ of μ_0 if $(\mathbf{a}, b) \in \mathcal{C}_0$ and μ_1 if $(\mathbf{a}, b) \in \mathcal{C}_{\frac{1}{2}}$.

- 1: $\bar{\mu} = \frac{\mu_0 + \mu_1}{2}$ and $\bar{\mu}' = \mu_0 - \bar{\mu}$
 - 2: $\text{testv} = (1 + X + \dots + X^{N-1}) \cdot X^{-\frac{N}{2}} \cdot \bar{\mu}'$
 - 3: $\bar{b} = \lfloor 2Nb \rfloor$ and $\bar{a}_i = \lfloor 2Na_i \rfloor$
 - 4: $\mathbf{c}' = \text{CP-BlindRotate}_{\text{testv}}((\bar{\mathbf{a}}, \bar{b}), \text{BK}, \text{RERAND})$
 - 5: $\mathbf{c} = (\mathbf{0}, \bar{\mu}) + \text{Extract}(\mathbf{c}')$
 - 6: Return $\text{KeySwitch}(\text{KS}, \mathbf{c})$
-

4.2 Sanitizing algorithm

We are now ready to present our sanitization algorithm. It is basically the **Bootstrap** algorithm from TFHE, with the test vector testv chosen to evaluate the identity function, and where **BlindRotate** is replaced with **CP-BlindRotate**. It is described more formally in Algorithm 2.

Lemma 16. *Algorithm 2 is sanitizing for the encryption scheme described in Section 2.6.*

Proof. The simulator $\text{Sim}_s(\mu, \text{BK}, \text{RERAND}, \text{KS})$ follows the same steps as the **Sanitize** algorithm except it computes $\mathbf{c}' = \text{Sim}_{cp}(\text{testv} \cdot X^\mu, \text{BK}, \text{RERAND})$, using the simulator Sim_{cp} of Lemma 14 instead of calling **CP-BlindRotate**.

To better understand why our statement stands, even though the input passed to Sim_{cp} does not seem to match the expected result from **CP-BlindRotate**, we need to further analyze **Extract** which is recalled in details in the supplementary materials, Section B.3 for the sake of completeness.

On input a RLWE ciphertext $(\mathbf{a} \parallel b)$, **Extract** trims the polynomial b to only its first coefficient $b(0)$, and rearranges the coefficients of $\mathbf{a}(X^{-1})$ into a vector. Recall that the output of $\text{Sim}_{cp}(\bar{\varphi})$ has the following distribution:

$$\text{CP-BlindRotate}_{\text{testv}}((\mathbf{0}, 0), \text{BK}, \text{PK}) + (\mathbf{0}, \text{testv} \cdot X^{\bar{\varphi}}).$$

As long as $\text{testv} \cdot X^{\bar{\varphi}} = \text{testv} \cdot X^{2N\mu}$, $\Delta(\text{Extract}(\text{Sim}_{cp}(\bar{\varphi})), \text{Extract}(\text{Sim}_{cp}(\mu))) = 0$. This is the case when $|\lfloor 2Nb \rfloor - \sum_{i=1}^n s_i \lfloor 2Na_i \rfloor - \mu| \leq \frac{N}{2}$, i.e. $(\mathbf{a}, b) \in \mathcal{C}_\mu$.

In order to prove the correctness property for the sanitizing algorithm, we analyze the distribution analogously to the analysis of the TFHE bootstrapping [CGGI20, Theorem 6.2]. We review the analysis of the keyswitching noise growth (Lemma 18) in Appendix B.4 for the sake of completeness.

Lemma 17 (Sanitization noise propagation). *If the parameters are set correctly, Algorithm 2 is message-space preserving. More precisely, we have that the variance of its output has a norm bounded by*

$$n2\pi q^2 \vartheta_{\square}(d+1) \ell N \vartheta_{\text{BK}} + \vartheta_{\text{RERAND}} + d \cdot N \left(\frac{\text{B}_{\text{KS}}^{-2t}}{4} + t \cdot \frac{\text{B}_{\text{KS}}^2}{4} \vartheta_{\text{KS}} \right),$$

where $\vartheta_{\text{RERAND}}$ is defined as in Lemma 15.

Proof. This results combines Lemma 15 and Lemma 18.

Security	N	ϑ_{BK}	n	ϑ_{KS}	B	ℓ	$\vartheta_{\square}$
80 bits	2048	$2^{-77.6}$	380	2^{-27}	2^5	8	$2^{-78.4}$
128 bits	1024	2^{-84}	700	$2^{-26.8}$	2^6	7	$2^{-83.4}$

Table 1. Parameters, security and secret keys and ciphertexts noise variances for different set of parameters.

5 Parameters, security and experimental results

In this section, we proceed to the selection of parameters for the CP-BlindRotate given in Algorithm 1.

Circuit Privacy: The parameters are selected with respect to a careful trade-off between efficiency and security. We summarize in Table 1 the variance parameters we took with respect to the following bounds:

- RGSW ciphertexts: the bootstrapping key BK is made of n RGSW ciphertexts of dimension $d = 1$ and N , with secret and noise both sampled from rounded continuous Gaussians with variance ϑ . The modulus is set as $q = B^\ell$.
- Corollary 1 gives a lower bound on the Gaussian parameter for $\mathbf{G}_r^{-1}()$. Taking the statistical advantage $\varepsilon = 2^{-80}$, we obtain $\vartheta_{\square}$ where we bound the norm of $\mathbf{e}$ in the condition of Corollary 1 using Lemma 3.
- PkEnc(PK) - public key: with $d' = d = 1$ and $\vartheta = \max\left(\vartheta_{\text{BK}}, \frac{1}{\pi}\eta_\varepsilon\left(\frac{1}{q}\mathbb{Z}^{(d'+d)N}\right)^2\right)$ in order to satisfy the conditions of Lemma 11, and rely on the same hardness assumption as the RGSW ciphertexts. We set $\vartheta_{\mathbf{r}} = \vartheta_{\mathbf{e}'} = 6\vartheta_{\text{BK}}$ and $\vartheta_{\mathbf{e}''} = 2\vartheta_{\text{BK}}N\left(\|\tilde{\mathbf{s}}\|_2^2 + q^2\|\mathbf{e}\|_2^2\right)$ to ensure minimal noise growth ϑ_{PK} while verifying the conditions of Lemma 11. We again bound the norms using Lemma 3.
- LWE ciphertexts: the keyswitching key KS is made of N LWE ciphertexts of dimension n and noise variance ϑ_{KS} , base decomposition $\mathbf{B}_{\text{KS}} = 4$ and precision $t = 6$.

Correctness: In order to decrypt correctly, the amplitude of the error needs to be $< \frac{1}{4}$ (rather than $\frac{1}{16}$ for the gate bootstrapping) with overwhelming probability. With the parameters derived from Lemma 17, we obtain an LWE ciphertext with error standard deviation σ , which corresponds to a probability of decryption failure below $\text{erfc}(\frac{1}{4\sqrt{2}\sigma})$. With the parameters chosen in Table 1, we obtain decryption failure of $2^{-89.6}$ and $2^{-96.5}$ respectively. We note that the majority of the noise comes from the keyswitching operation.

Security: For LWE, RLWE and RGSW ciphertexts we use, the security is estimated using commit 5ba00f5 of the Lattice Estimator [APS15] to 81.7 bits for the first set of parameters and 143.2 bits for the second one. The bottleneck for security is the LWE assumption underlying the keyswitching key. The RLWE assumption underlying the bootstrapping key is estimated at 159.9 bits of security.

Implementation: As a proof of concept of our technique, we provide a C implementation compatible with Windows, Linux, and MacOS and which is accessible from <https://github.com/fhe-extension/fhe-sanitize>. We ran multiple experiments using the parameters described previously and provide the results in Table 2. The randomness is sampled using the CSPRNG of windows or the one from OpenSSL. We use the library

Security -	Bootstrap -	Sanitize w/ pre-comp.	Sanitize w/o pre-comp.	PkEnc(PK) -	Multithread -
80 bits	0.3 s	0.38 s	5 s	0.4 ms	2.5 s
128 bits	0.99 s	1.25 s	21.2 s	1.2 ms	4.5 s

Table 2. Timings of our concrete implementation. The experiments were run on a AMD Ryzen 7 PRO 8840HS laptop. The first column reports the timing of our implementation of the standard TFHE bootstrapping, the second one reports the timing of our sanitization algorithm using pre-computed discrete Gaussian samples for $\mathbf{G}_r^{-1}(\cdot)$, the third one reports the timings including the timings of Gaussian sampling computation. The last column gives the timing of generating and adding a randomizer to a ciphertext using our new public key encryption scheme.

FFTW⁵ for our Fourier transformation. We implement the sampler for $\mathbf{G}_r^{-1}(\cdot)$ using the Gaussian sampler from [GPV08,MP12] and implement the inversion sampling for the base sampler from [MW17]. Since our goal is to evaluate the overhead of our sanitization technique, we compare the non-sanitizing bootstrapping (first column of Table 2) and the sanitizing bootstrapping (remaining columns of Table 2) with the same parameters on the same laptop, with the same level of optimizations⁶.

We found that generating small Gaussian samples with our base sampler accounts for 80% of the total execution time of our implementation. We have decided to implement precomputation for these samples to assess the efficiency of our strategy with access to specialized hardware for Gaussian sampling, or to determine if these samples could be computed in parallel or before the online sanitization phase. In that case, sanitizing a FHE ciphertext with our strategy takes around 25% longer than a non-sanitizing bootstrapping, which is a relatively small overhead. To further emphasize this point, we also implemented multithreading using a producer-consumer framework to generate base Gaussian samples over $\mathbb{Z}$ in separate threads. Our experiments show that the runtime can be reduced to 2.5 s for 80 bits of security and 4.5 s for 128 bits of security, using mutexes and semaphores for synchronisation.

The discussion in appendix A shows that using a similar set of parameters, 9 cycles of the soak-and-spin strategy from [DS16] instantiated with our new public key encryption scheme would be required to reach the same level of security. Extrapolating our timings of the bootstrapping, this strategy would take around 9.5 s to sanitize a ciphertext.

6 Conclusion

We present three sanitization strategies each tailored to a specific scenario and to distinct parameter trade-offs:

- when large parameters are required, such situation could arise either because a large message space has to be supported or a large security is required:
sanitization can be reached by replacing deterministic gadget decomposition with randomized gadget decomposition, without the need for a public key encryption scheme for randomization;
- assuming Gaussian sampling can be executed efficiently, such situation could arise when specialized hardware can be used or when multiple threads can run in parallel

⁵ from <https://www.fftw.org/>.

⁶ This explains our choice of implementing TFHE from scratch rather than using an existing implementation of the bootstrapping.

to sample discrete Gaussian or because there is downtime between online sessions to precompute the samples:

sanitization can be reached by replacing deterministic gadget decomposition with randomized gadget decomposition and adding a randomizer using our public key encryption scheme with small public key;

- when Gaussian sampling is too expensive, such situation could arise either because the entropy available is very small the program must reply continually in real time, or when the program only runs on a single core:

even though Gaussian sampling is too expansive, our work still provides a very large efficiency improvement by implementing the soak-and-spin strategy of [DS16] with our new public key randomizer.

References

- AGHS13. Shweta Agrawal, Craig Gentry, Shai Halevi, and Amit Sahai. Discrete Gaussian leftover hash lemma over infinite domains. In Kazue Sako and Palash Sarkar, editors, *ASIACRYPT 2013, Part I*, volume 8269 of *LNCS*, pages 97–116. Springer, Berlin, Heidelberg, December 2013. doi:10.1007/978-3-642-42033-7_6.
- AP14. Jacob Alperin-Sheriff and Chris Peikert. Faster bootstrapping with polynomial error. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 297–314. Springer, Berlin, Heidelberg, August 2014. doi:10.1007/978-3-662-44371-2_17.
- AP20. Marc Abboud and Thomas Prest. Cryptographic divergences: New techniques and new applications. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 492–511. Springer, Cham, September 2020. doi:10.1007/978-3-030-57990-6_24.
- APS15. Martin R. Albrecht, Rachel Player, and Sam Scott. On the concrete hardness of learning with errors. *Journal of Mathematical Cryptology. Volume 9, Issue 3, Pages 169–203, ISSN (Online) 1862-2984*, October 2015.
- ASY22. Shweta Agrawal, Damien Stehlé, and Anshu Yadav. Round-optimal lattice-based threshold signatures, revisited. In Mikolaj Bojanczyk, Emanuela Merelli, and David P. Woodruff, editors, *ICALP 2022*, volume 229 of *LIPICs*, pages 8:1–8:20. Schloss Dagstuhl, July 2022. doi:10.4230/LIPICs.ICALP.2022.8.
- Ban93. Wojciech Banaszczyk. New bounds in some transference theorems in the geometry of numbers. *Annals of Mathematics*, 296(4):625–635, 1993.
- BDF18. Guillaume Bonnoron, Léo Ducas, and Max Fillinger. Large FHE gates from tensored homomorphic accumulator. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18*, volume 10831 of *LNCS*, pages 217–251. Springer, Cham, May 2018. doi:10.1007/978-3-319-89339-6_13.
- BdPMW16. Florian Bourse, Rafaël del Pino, Michele Minelli, and Hoeteck Wee. FHE circuit privacy almost for free. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part II*, volume 9815 of *LNCS*, pages 62–89. Springer, Berlin, Heidelberg, August 2016. doi:10.1007/978-3-662-53008-5_3.
- BGG⁺18. Dan Boneh, Rosario Gennaro, Steven Goldfeder, Aayush Jain, Sam Kim, Peter M. R. Rasmussen, and Amit Sahai. Threshold cryptosystems from threshold fully homomorphic encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 565–596. Springer, Cham, August 2018. doi:10.1007/978-3-319-96884-1_19.
- BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) fully homomorphic encryption without bootstrapping. In Shafi Goldwasser, editor, *ITCS 2012*, pages 309–325. ACM, January 2012. doi:10.1145/2090236.2090262.
- BLP⁺13. Zvika Brakerski, Adeline Langlois, Chris Peikert, Oded Regev, and Damien Stehlé. Classical hardness of learning with errors. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th ACM STOC*, pages 575–584. ACM Press, June 2013. doi:10.1145/2488608.2488680.
- Bra12. Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 868–886. Springer, Berlin, Heidelberg, August 2012. doi:10.1007/978-3-642-32009-5_50.

- BV14. Zvika Brakerski and Vinod Vaikuntanathan. Lattice-based fhe as secure as pke. In Moni Naor, editor, *ITCS*, pages 1–12. ACM, 2014.
- CGGI20. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: Fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, January 2020. doi:10.1007/s00145-019-09319-x.
- CHK⁺18. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for approximate homomorphic encryption. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 360–384. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78381-9_14.
- CIM19. Sergiu Carpov, Malika Izabachène, and Victor Mollimard. New techniques for multi-value input homomorphic evaluation and applications. In Mitsuru Matsui, editor, *CT-RSA 2019*, volume 11405 of *LNCS*, pages 106–126. Springer, Cham, March 2019. doi:10.1007/978-3-030-12612-4_6.
- CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *ASIACRYPT 2017, Part I*, volume 10624 of *LNCS*, pages 409–437. Springer, Cham, December 2017. doi:10.1007/978-3-319-70694-8_15.
- CLOT21. Ilaria Chillotti, Damien Ligier, Jean-Baptiste Orfila, and Samuel Tap. Improved programmable bootstrapping with larger precision and efficient arithmetic circuits for TFHE. In *Asiacrypt*, LNCS 13092, pages 670–699. Springer-Verlag, 2021.
- CLR17. Hao Chen, Kim Laine, and Peter Rindal. Fast private set intersection from homomorphic encryption. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1243–1255. ACM Press, October / November 2017. doi:10.1145/3133956.3134061.
- CMdG⁺21. Kelong Cong, Radames Cruz Moreno, Mariana Botelho da Gama, Wei Dai, Ilia Iliashenko, Kim Laine, and Michael Rosenberg. Labeled psi from homomorphic encryption with reduced computation and communication. In *CCS 2021*, pages 1135–1150. ACM, 2021. URL: eprint2021/116.
- CZ17. Long Chen and Zhenfeng Zhang. Bootstrapping fully homomorphic encryption with ring plaintexts within polynomial noise. In Tatsuaki Okamoto, Yong Yu, Man Ho Au, and Yinnan Li, editors, *ProvSec 2017*, volume 10592 of *LNCS*, pages 285–304. Springer, Cham, October 2017. doi:10.1007/978-3-319-68637-0_18.
- DGKS21. Dana Dachman-Soled, Huijing Gong, Mukul Kulkarni, and Aria Shahverdi. Towards a ring analogue of the leftover hash lemma. *J. Math. Cryptol.*, 15(1):87–110, 2021.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 617–640. Springer, Berlin, Heidelberg, April 2015. doi:10.1007/978-3-662-46800-5_24.
- DS16. Léo Ducas and Damien Stehlé. Sanitization of FHE ciphertexts. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part I*, volume 9665 of *LNCS*, pages 294–310. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49890-3_12.
- FV12. Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *Cryptology ePrint Archive*, Report 2012/144, 2012.
- GBA21. Antonio Guimarães, Edson Borin, and Diego F. Aranha. Revisiting the functional bootstrap in TFHE. *IACR TCHES*, 2021(2):229–253, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/8793>, doi:10.46586/tches.v2021.i2.229-253.
- Gen09. Craig Gentry. Fully homomorphic encryption using ideal lattices. In Michael Mitzenmacher, editor, *41st ACM STOC*, pages 169–178. ACM Press, May / June 2009. doi:10.1145/1536414.1536440.
- GHV10. Craig Gentry, Shai Halevi, and Vinod Vaikuntanathan. i-Hop homomorphic encryption and rerandomizable Yao circuits. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 155–172. Springer, Berlin, Heidelberg, August 2010. doi:10.1007/978-3-642-14623-7_9.
- GPV08. Craig Gentry, Chris Peikert, and Vinod Vaikuntanathan. Trapdoors for hard lattices and new cryptographic constructions. In Richard E. Ladner and Cynthia Dwork, editors, *40th ACM STOC*, pages 197–206. ACM Press, May 2008. doi:10.1145/1374376.1374407.
- GPvL23. Antonio Guimarães, Hilder V. L. Pereira, and Barry van Leeuwen. Amortized bootstrapping revisited: Simpler, asymptotically-faster, implemented. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VI*, volume 14443 of *LNCS*, pages 3–35. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8736-8_1.

- GSW13. Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 75–92. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-40041-4_5.
- HMS25. Intak Hwang, Seonhong Min, and Yongsoo Song. Practical circuit privacy/sanitization for TFHE. Cryptology ePrint Archive, Report 2025/216, to appear at CCS 2025, 2025. URL: <https://eprint.iacr.org/2025/216>.
- KLSS23. Duhyeon Kim, Dongwon Lee, Jinyeong Seo, and Yongsoo Song. Toward practical lattice-based proof of knowledge from hint-mlwe. In *43rd Annual International Cryptology Conference, CRYPTO 2023, Santa Barbara, CA, USA, August 20-24, 2023, Proceedings, Part V*, volume 14085 of *Lecture Notes in Computer Science*, pages 549–580. Springer, 2023.
- Klu22. Kamil Kluczniak. Circuit privacy for FHEW/TFHE-style fully homomorphic encryption in practice. Cryptology ePrint Archive, Report 2022/1459, 2022. URL: <https://eprint.iacr.org/2022/1459>.
- Klu24. Kamil Kluczniak. Circuit privacy for FHEW/TFHE-style fully homomorphic encryption in practice. *CiC*, 1(4):33, 2024. doi:10.62056/av11c3w9p.
- LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On ideal lattices and learning with errors over rings. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 1–23. Springer, Berlin, Heidelberg, May / June 2010. doi:10.1007/978-3-642-13190-5_1.
- LPR13. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-LWE cryptography. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013. doi:10.1007/978-3-642-38348-9_3.
- LW20. Feng-Hao Liu and Zhedong Wang. Rounding in the rings. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 296–326. Springer, Cham, August 2020. doi:10.1007/978-3-030-56880-1_11.
- LY23. KangHoon Lee and Ji Won Yoon. Discretization error reduction for high precision torus fully homomorphic encryption. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part II*, volume 13941 of *LNCS*, pages 33–62. Springer, Cham, May 2023. doi:10.1007/978-3-031-31371-4_2.
- MKMS22. Jose Maria Bermudo Mera, Angshuman Karmakar, Tilen Marc, and Azam Soleimanian. Efficient lattice-based inner-product functional encryption. In Goichiro Hanaoka, Junji Shikata, and Yohei Watanabe, editors, *PKC 2022, Part II*, volume 13178 of *LNCS*, pages 163–193. Springer, Cham, March 2022. doi:10.1007/978-3-030-97131-1_6.
- MM11. Daniele Micciancio and Petros Mol. Pseudorandom knapsacks and the sample complexity of LWE search-to-decision reductions. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 465–484. Springer, Berlin, Heidelberg, August 2011. doi:10.1007/978-3-642-22792-9_26.
- MP12. Daniele Micciancio and Chris Peikert. Trapdoors for lattices: Simpler, tighter, faster, smaller. In David Pointcheval and Thomas Johansson, editors, *EUROCRYPT 2012*, volume 7237 of *LNCS*, pages 700–718. Springer, Berlin, Heidelberg, April 2012. doi:10.1007/978-3-642-29011-4_41.
- MP21. Daniele Micciancio and Yuriy Polyakov. Bootstrapping in fhew-like cryptosystems. In *WAHC '21: Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Virtual Event, Korea, 15 November 2021*, pages 17–28. ACM, 2021.
- MR07. Daniele Micciancio and Oded Regev. Worst-case to average-case reductions based on gaussian measures. *SIAM J. Comput.*, 37(1):267–302, 2007.
- MS18. Daniele Micciancio and Jessica Sorrell. Ring packing and amortized FHEW bootstrapping. In Ioannis Chatzigiannakis, Christos Kallamanis, Daniel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPICs*, pages 100:1–100:14. Schloss Dagstuhl, July 2018. doi:10.4230/LIPICs.ICALP.2018.100.
- MW17. Daniele Micciancio and Michael Walter. Gaussian sampling over the integers: Efficient, generic, constant-time. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part II*, volume 10402 of *LNCS*, pages 455–485. Springer, Cham, August 2017. doi:10.1007/978-3-319-63715-0_16.
- OPP14. Rafail Ostrovsky, Anat Paskin-Cherniavsky, and Beni Paskin-Cherniavsky. Maliciously circuit-private FHE. In Juan A. Garay and Rosario Gennaro, editors, *CRYPTO 2014, Part I*, volume 8616 of *LNCS*, pages 536–553. Springer, Berlin, Heidelberg, August 2014. doi:10.1007/978-3-662-44371-2_30.
- Pei10. Chris Peikert. An efficient and parallel Gaussian sampler for lattices. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 80–97. Springer, Berlin, Heidelberg, August 2010. doi:10.1007/978-3-642-14623-7_5.

- Reg05. Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005. doi:10.1145/1060590.1060603.
- SSTX09. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient public key encryption based on ideal lattices. In Mitsuru Matsui, editor, *ASIACRYPT 2009*, volume 5912 of *LNCS*, pages 617–635. Springer, Berlin, Heidelberg, December 2009. doi:10.1007/978-3-642-10366-7_36.

Supplementary material

A A new instantiation of [DS16]’s sanitization strategy

In this section, we evaluate the cost of sanitizing a TFHE ciphertext using the strategy proposed in [DS16] with a parameter set similar to ours.

Recall that [DS16] sanitization algorithm consists of the iterations of two steps: (1) a refresh step which, given an **LWE** ciphertext as input, brings the noise down to a low level; (2) a randomization step which randomizes a ciphertext by adding public key encryption of 0 and the addition of a uniform small noise to decrease the statistical distance between two hypothetical ciphertexts.

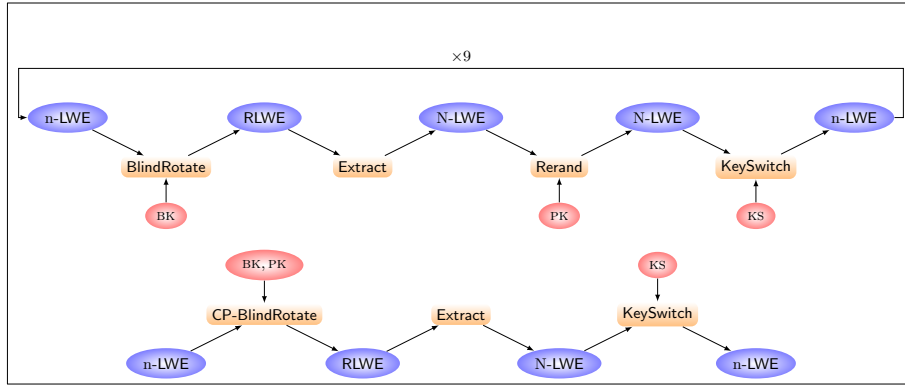


Fig. 2. High-level overview of our sanitization strategy (bottom part) and [DS16] strategy (top part).

Our parameters were set with the idea that sanitization is the last step of computation, so we only considered correctness for decryption, and not for being allowed as input of a bootstrapping. However, if we want to apply the soak-and-spin paradigm, the output of one refresh plus a rerand step needs a noise small enough to be bootstrapped, that is, we need to take into account the noise coming from modulus switching before decryption.

In order to account for that, we slightly modified the parameters for the keyswitching in order to decrease the noise of the output ciphertext. Taking $n = 750$ and $\vartheta_{KS} = 2^{-28.4}$, the security level remains estimated at 145.8 bits, and the bootstrapping time is brought to 1.06 s.

As done in the [DS16] analysis applied to FHEW, for maximal efficiency, the randomization step is done right after the **Extract** but before switching to a lower dimension. The number of cycles in [DS16] strategy varies depending on the FHE scheme and the parameters. This number is computed from the magnitude of the noise of the current ciphertext compared to the magnitude of the noise introduced by **RERAND**.

With the chosen parameters, the noise in a ciphertext obtained by **BlindRotate** and **RERAND**, before adding the soaking noise is bounded by $2^{-14.6}$ with probability lower than $2^{-83.5}$ (using the **erfc** function again). We also show that adding a soaking noise uniform bounded by $2^{5.6}$ still yields decryption with probability around $2^{-84.7}$ for each

bootstrapping, accounting for the modulus switching noise with bound $\frac{n}{4N}$.

Thus, each cycle reaches a statistical distance of $\frac{2^{-14.6}}{2^{-5.6}} = 2^{-9}$, so 9 cycles is enough to reach a statistical distance of 2^{-80} .

Figure 2 gives a high-level overview of both strategies to highlight the differences between them: assuming we perform the same randomization step, we perform them at each step of the **BlindRotate** algorithm, whereas the soak-and-spin strategy applies it only once per iteration of the bootstrapping, but requires 9 such iterations to reach the same level of circuit privacy security.

B TFHE Bootstrapping building blocks

B.1 $\mathbf{G}^{-1}(\cdot)$ computation

In Algorithm 3, we recall the $\mathbf{G}^{-1}$ algorithm that decomposes vectors in $\hat{R}_q^{1 \times (d+1)}$ into vectors in $R^{1 \times (d+1) \cdot \ell}$.

Algorithm 3 Gadget Decomposition

Input: A vector $\mathbf{v} = (v_1 \mid \dots \mid v_{d+1}) \in \hat{R}_q^{1 \times (d+1)}$

Output: A vector $\mathbf{x}$ such that $\mathbf{x} \cdot \mathbf{G} = \mathbf{v}$ and $\|\mathbf{x}\|_\infty \leq B/2$

- 1: For each v_i choose the unique representative $\sum_{j=0}^{N-1} v_{i,j} X^j$, with $v_{i,j} \in \hat{\mathbb{Z}}_q$. Note that $v_{i,j}$ is an exact multiple of $\frac{1}{B^\ell}$.
 - 2: Decompose each $v_{i,j}$ uniquely as $\sum_{p=1}^{\ell} v_{i,j,p} \frac{1}{B^p}$ where each $v_{i,j,p} \in [-B/2, B/2[$
 - 3: **for** $i = 1$ **to** $d + 1$
 - 4: **for** $p = 1$ **to** ℓ
 - 5: $x_{i,p} = \sum_{j=0}^{N-1} v_{i,j,p} X^j \in R$
 - 6: **Return** $(x_{1,1} \mid \dots \mid x_{d+1,\ell})$
-

B.2 $\mathbf{G}_r^{-1}(\cdot)$ computation

In Algorithm 4, we recall the $\mathbf{G}_r^{-1}$ algorithm that decomposes vectors in $\hat{R}_q^{1 \times (d+1)}$ into vectors in $R^{1 \times (d+1) \cdot \ell}$ with spherical Gaussian distribution on a coset of $\Lambda^\perp(\mathbf{G})$.

B.3 Extract

In Algorithm 5, we recall the **Extract** algorithm that transforms RLWE ciphertexts into N -LWE ciphertexts.

B.4 KeySwitch

In Algorithm 6, we recall the keyswitching procedure of the bootstrapping, and the analysis of noise growth is provided in Lemma 18

Algorithm 4 Randomized Gadget Decomposition**Input:** A vector $\mathbf{v} = (v_1 \mid \dots \mid v_{d+1}) \in \hat{R}_q^{1 \times (d+1)}$ **Output:** A vector $\mathbf{x}$ from a spherical Gaussian distribution with parameter r such that $x \cdot \mathbf{G} = \mathbf{v}$

- 1: For each v_i choose the unique representative $\sum_{j=0}^{N-1} v_{i,j} X^j$, with $v_{i,j} \in \hat{\mathbb{Z}}_q$. Note that $v_{i,j}$ is an exact multiple of $\frac{1}{B^\ell}$.
- 2: Set $\mathbf{v}_j = (v_{1,j} \mid \dots \mid v_{d+1,j})$
- 3: For each j sample $\mathbf{x}_j = (x_{1,j}, \dots, x_{(d+1) \cdot \ell, j}) \in R^{(d+1) \cdot \ell}$ from $\Lambda_{\mathbf{v}_j}^\perp(\mathbf{G})$ with parameter r
- 4: **for** $i = 1$ **to** $(d+1)\ell$
- 5: $x_i = \sum_{j=0}^{N-1} x_{i,j} X^j \in R$
- 6: **Return** $(x_1 \mid \dots \mid x_{(d+1) \cdot \ell})$

Algorithm 5 Extracting an N -LWE ciphertext of the constant term from a RLWE ciphertext**Input:** $\mathbf{c} = (\mathbf{a}, b) \in \hat{R}_q^{d+1}$ **Output:** Extract($\mathbf{c}$): an N -LWE encryption of $\varphi_{\mathbf{s}}(\mathbf{c})$.

- 1: $b' = b(0)$.
- 2: **for** $i = 1$ **to** d
- 3: Set $(a'_{iN+j})_{j \in [0, N-1]}$ such that $\sum_{j=0}^{N-1} a'_{iN+j} X^j = a_i(\frac{1}{X})$.
- 4: **Return** $\mathbf{c}' = (\mathbf{a}', b')$.

Lemma 18 (Algorithm 6 noise propagation). Let $\mathbf{c} \in \hat{\mathbb{Z}}_q^{dN+1}$ with noise variance $\vartheta_{\mathbf{c}}$, $\mathbf{KS} = (\mathbf{KS}_{i,j})_{i,j}$, where $\mathbf{KS}_{i,j}$ is an n -LWE encryption of $K_i \frac{1}{\mathbf{B}_{\mathbf{KS}}^j}$ for $i \in [1, dN]$ and $j \in [1, t]$ with noise variance $\vartheta_{\mathbf{KS}}$. Algorithm 6 outputs a sample $\mathbf{c}' \in n\text{-LWE}_{\mathbf{s}}(\varphi_K(\mathbf{c}))$ such that:

$$\text{Var}(\mathbf{c}') \leq \vartheta_{\mathbf{c}} + d \frac{tN \mathbf{B}_{\mathbf{KS}}^2}{4} \vartheta_{\mathbf{KS}} + \frac{\|K\|^2}{4 \mathbf{B}_{\mathbf{KS}}^{2t}} \quad (5)$$

Proof. During the first step of the algorithm, each rounding induces an error of at most $\frac{1}{2\mathbf{B}_{\mathbf{KS}}^t}$, hence the resulting variance for dN roundings.

Each part of the keyswitching key is multiplied by a coefficient between $-\frac{\mathbf{B}_{\mathbf{KS}}}{2}$ and $\frac{\mathbf{B}_{\mathbf{KS}}}{2}$, hence the resulting variance for combining dtN of them.

Algorithm 6 keyswitching from secret key K of dimension dN to secret key $\mathbf{s}$ of dimension n

Input: $\mathbf{c} = (\mathbf{a}, b) \in \hat{\mathbb{Z}}_q^{dN+1}$, a keyswitching key $\text{KS} = (\text{KS}_{i,j})_{i,j}$, where $\text{KS}_{i,j}$ is an n -LWE encryption under secret key $\mathbf{s}$ of $K_i \frac{1}{\mathbf{B}_{\text{KS}}^j}$ for $i \in [1, dN]$ and $j \in [1, t]$.

Output: an n -LWE encryption of $\varphi_K(\mathbf{c})$.

- 1: **for** $i = 1$ **to** dN
- 2: Round a_i to the nearest element $\lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t}$ in $\frac{1}{\mathbf{B}_{\text{KS}}^t} \mathbb{Z}$.
- 3: Set $(a_{i,j})_{j \in [1, t]}$ such that $\sum_{j=1}^t a_{i,j} \frac{1}{\mathbf{B}_{\text{KS}}^j} = \lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t}$ (Decompose $\lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t}$ in basis $\mathbf{B}_{\text{KS}}$).
- 4: Return $\mathbf{c}' = (\mathbf{0}, b) - \sum_{i,j} a_{i,j} \text{KS}_{i,j}$ where the sum ranges over $[1, dN] \times [1, t]$.

Given $(\mathbf{a}, b)$ a LWE ciphertext under secret key K in input, we have:

$$\begin{aligned}
 \varphi_{\mathbf{s}}(\mathbf{c}') &= \varphi_{\mathbf{s}}(\mathbf{0}, b) - \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \varphi_{\mathbf{s}}(\text{KS}_{i,j}) \\
 &= b - \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \left(\mathbf{B}_{\text{KS}}^{-j} K_i + \text{Err}(\text{KS}_{i,j}) \right) \\
 &= b - \sum_{i=1}^{dN} a_{i,j} \mathbf{B}_{\text{KS}}^{-j} K_i + \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \text{Err}(\text{KS}_{i,j}) \\
 &= b - \sum_{i=1}^{dN} \lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t} K_i - \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \text{Err}(\text{KS}_{i,j}) \\
 &= b - \sum_{i=1}^{dN} a_i K_i - \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \text{Err}(\text{KS}_{i,j}) + \sum_{i=1}^{dN} (a_i - \lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t}) K_i \\
 &= \varphi_K(\mathbf{a}, b) - \sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \text{Err}(\text{KS}_{i,j}) + \sum_{i=1}^{dN} (a_i - \lfloor a_i \rfloor_{\mathbf{B}_{\text{KS}}^t}) K_i
 \end{aligned}$$

As the $\text{KS}_{i,j}$'s and K_i 's are fresh and independent samples, we have:

$$\begin{aligned}
 \text{Var}(\mathbf{c}') &\leq \vartheta_{\mathbf{c}} + \text{Var} \left(\sum_{i=1}^{dN} \sum_{j=1}^t a_{i,j} \text{KS}_{i,j} \right) + \frac{\|K\|_2^2}{4\mathbf{B}_{\text{KS}}^{2t}} \\
 &\leq \vartheta_{\mathbf{c}} + \frac{tdN\mathbf{B}_{\text{KS}}^2}{4} \vartheta_{\text{KS}} + \frac{\|K\|_2^2}{4\mathbf{B}_{\text{KS}}^{2t}}
 \end{aligned}$$

C Proof of Lemma 9

In this section, we will require the following lemma :

Lemma 19 ([Reg05, Claim 3.8]). *Let $\Lambda \subseteq \mathbb{Z}^m$ be any lattice, $\mathbf{c} \in \mathbb{R}^m$, $\varepsilon > 0$ and $r \geq \eta_{\varepsilon}(\Lambda)$. Then*

$$\rho_r(\Lambda + \mathbf{c}) \in \frac{r^m}{\det(\Lambda)(1 \pm \varepsilon)}$$

Before heading into the proof, we first characterize the underlying structure of the elements used in our scheme.

Structure of R_q . First, let us discuss the structure of R_q , where $q = 2^k$. Since $X^N + 1 = (X + 1)^N \pmod{2}$, the ideals of R_q are generated by $X + 1$ and 2, so any ideal of R_q can be written as $2^i(X + 1)^j R_q$, for some $i \in [0, k - 1]$ and some $j \in [0, N - 1]$. Moreover, we have that in R_q , $(X + 1)^N = 2Q(X)$, with Q an invertible polynomial in R_q . This follows from the following three facts:

- First, $(X + 1)^N = 1 + 2X^{\frac{N}{2}} + X^N \pmod{4}$; this can be easily shown by induction (recall that N is a power of 2).
- Second, for any polynomial $P \in R_q$, P is invertible if and only if $P \pmod{X + 1}$ is odd. If d is odd, $((X + 1)C(X) + d)^{-1} = \frac{1}{d} \sum_{i=0}^{N-1} \left(-\frac{(X+1)C(X)}{d} \right)^i$.
- Third and last, the parity of $P \pmod{X + 1}$ is the same as the parity of the sum of the coefficients of P .

Combining all of these, we have that $X^{\frac{N}{2}}$ is invertible, so $(X + 1)^N R_q = 2R_q$, which gives us the following alternative definition for the ideals of R_q : any ideal of R_q is of the form $I_t = (X + 1)^t R_q$ for some $t \in [0, Nk]$. We also have the following particular cases :

$$\begin{aligned} I_0 &= R_q \\ I_1 &= R_q^\times = \left\{ P = \sum_{i=0}^{N-1} a_i X^i \mid \sum_{i=0}^{N-1} a_i = 0 \pmod{2} \right\} \\ I_{Nk} &= \{0\} \end{aligned}$$

We also have that any polynomial $P \in R_q$ can be uniquely described as

$$P = \sum_{i=0}^{Nk-1} p_i (X + 1)^i, \text{ with } p_i \in \{0, 1\}.$$

In the following, we will use $P \pmod{(X + 1)^j}$ to denote the truncated sum $\sum_{i=0}^{j-1} p_i (X + 1)^i$, so $P \pmod{X + 1}$ is actually $P \pmod{2} \pmod{X + 1}$.

For a set of m polynomials $\mathbf{P} = (P_1, P_2, \dots, P_m) \in R_q^m$, we define their gcd in the following natural way:

$$\gcd(\mathbf{P}) = \max \{t \in \{0, \dots, Nk\} \mid \forall i \in \{1, \dots, m\}, P_i \in I_t\}$$

Fixing any vector $\mathbf{v} = (v_1, v_2, \dots, v_m) \in I_t^m$ such that $\gcd(\mathbf{v}) = t$, there exists $i \in \{1, \dots, m\}$ such that $v_i \in I_t \setminus I_{t+1}$, meaning that $v_i = (X + 1)^t Q$, with Q an invertible polynomial in R_q .

Fixing any values $a_j \in \hat{R}_q$, for all $j \neq i$, we have

$$\begin{aligned} \mathbf{v}^t \mathbf{a} = \mathbf{0} &\Leftrightarrow v_i a_i = - \sum_{j \neq i} v_j a_j \\ &\Leftrightarrow a_i (X + 1)^t = -Q^{-1} \sum_{j \neq i} v_j a_j \\ &\Leftrightarrow a_i = -Q^{-1} \left(\sum_{j \neq i} v_j a_j \right) / (X + 1)^t \pmod{(X + 1)^{Nk-t}} \end{aligned}$$

This give us the following useful fact:

$$\Pr [\mathbf{v}^t \mathbf{a} = \mathbf{0}] = \frac{1}{2^{Nk-t}}, \quad (6)$$

where $\mathbf{a}$ is uniform over $\hat{R}_q^m$.

We are now ready to prove Lemma 9.

Proof. We adapt the proof of [MM11, Lemma 2.3] and [LW20, Theorem 5.5] to our setting. For any distribution $\mathcal{Z}$ over a set Z , we have the following relation between the statistical distance between $\mathcal{Z}$ and the uniform distribution over Z , $\mathcal{U}_Z$, with the collision probability of $\mathcal{Z}$, $\text{Col}(\mathcal{Z})$.

$$\Delta(\mathcal{Z}, \mathcal{U}_Z) \leq \frac{1}{2} \sqrt{|Z| \cdot \text{Col}(\mathcal{Z}) - 1}.$$

We are interested in the distance to uniformity of $(\mathbf{A}, \mathbf{x}^t \cdot \mathbf{A})$ conditioned on $\mathbf{e}^t \mathbf{x} + \mathbf{y}$, where $\mathbf{A}$ is uniform over $\hat{R}_q^{(d+1) \cdot \ell \times d}$ and $\mathbf{x} \leftarrow \mathcal{D}_{\Lambda^\perp(\mathbf{G})+\mathbf{c}, r}$, $\mathbf{c} \in R_q^{(d+1) \cdot \ell}$. The collision probability can be rewritten as follow:

$$\begin{aligned} \Pr [\mathbf{A} = \mathbf{A}' \wedge \mathbf{x}^t \mathbf{A} = \mathbf{x}'^t \mathbf{A}'] &= \Pr [\mathbf{A} = \mathbf{A}'] \cdot \Pr [\mathbf{x}^t \mathbf{A} = \mathbf{x}'^t \mathbf{A}' \mid \mathbf{A} = \mathbf{A}'] \\ &= q^{-Nd(d+1) \cdot \ell} \Pr [(\mathbf{x}^t - \mathbf{x}'^t) \mathbf{A} = \mathbf{0}] \end{aligned}$$

$(\mathbf{A}, \mathbf{x}^t \cdot \mathbf{A})$ is a random variable over $\hat{R}_q^{(d+1) \cdot \ell \times d} \times \hat{R}_q^d$, of size $q^{Nd(d+1) \cdot \ell} \cdot q^{Nd}$. So its distance to uniformity is bounded by

$$\frac{1}{2} \sqrt{q^{Nd} \Pr [(\mathbf{x}^t - \mathbf{x}'^t) \mathbf{A} = \mathbf{0}] - 1} \quad (7)$$

In order to bound $\Pr [(\mathbf{x}^t - \mathbf{x}'^t) \mathbf{A} = \mathbf{0}]$, we use the law of total probability conditioning on the value of $\text{gcd}(\mathbf{x} - \mathbf{x}')$.

Using d times the identity (6), once for each column of $\mathbf{A}$ which are sampled independently, we have

$$\Pr [(\mathbf{x}^t - \mathbf{x}'^t) \mathbf{A} = \mathbf{0} \mid \text{gcd}(\mathbf{x} - \mathbf{x}') = t] = 2^{-d(Nk-t)} = q^{-Nd} 2^{dt}$$

The bound (7) becomes

$$\begin{aligned} & \frac{1}{2} \sqrt{q^{Nd} \sum_{t=0}^{Nk} q^{-Nd} 2^{dt} \Pr [\text{gcd}(\mathbf{x} - \mathbf{x}') = t] - 1} \\ &= \frac{1}{2} \sqrt{\sum_{t=0}^{Nk} 2^{dt} \Pr [\text{gcd}(\mathbf{x} - \mathbf{x}') = t] - 1} \\ &\leq \frac{1}{2} \sqrt{\sum_{t=1}^{Nk} 2^{dt} \Pr [\text{gcd}(\mathbf{x} - \mathbf{x}') = t]} \end{aligned}$$

Now, notice that for all t , $I_{t+1} \subset I_t$, so

$$\begin{aligned}
\Pr [\gcd(\mathbf{x} - \mathbf{x}' = t)] &= \Pr [\mathbf{x} - \mathbf{x}' \in I_t^{(d+1) \cdot \ell}] - \Pr [\mathbf{x} - \mathbf{x}' \in I_{t+1}^{(d+1) \cdot \ell}] \\
&\leq \Pr [\mathbf{x} - \mathbf{x}' \in I_t] \\
&= \Pr [\mathbf{x} \equiv \mathbf{x}' \pmod{(X+1)^t}] \\
&= \text{Col}(\mathcal{X}_t) \\
&\leq \text{Col}(\mathcal{X}_1) \\
&\leq 2^{-H_\infty(\mathbf{x} \pmod{(X+1)} \mid \mathbf{e}^t \mathbf{x} + \mathbf{y})}
\end{aligned}$$

where $\text{Col}(\mathcal{X}_t)$ is the collision probability of the distribution $\mathbf{x} \pmod{(X+1)^t}$ conditioned on $\mathbf{e}^t \mathbf{x} + \mathbf{y}$. In order to bound the collision probability $\text{Col}(\mathcal{X}_t)$, we use the min-entropy of $\mathbf{x} \pmod{(X+1)}$ conditioned on $\mathbf{e}^t \mathbf{x} + \mathbf{y}$.

We are thus interested in the weights of cosets of the lattices $(\Lambda^\perp(\mathbf{G}) + \mathbf{c})$ and its sublattice $\mathcal{L}$ composed of vectors of polynomials whose coordinates have the sum of their coefficients even.

The former one has determinant $B^{N(d+1) \cdot \ell}$ because it is generated by the basis $\mathbf{I}_{d+1} \otimes \mathbf{B}_0 \otimes \mathbf{I}_N$, with

$$\mathbf{B}_0 = \begin{pmatrix} B-1 & & & \\ & \ddots & \ddots & \\ & & \ddots & -1 \\ & & & B \end{pmatrix}$$

The latter has determinant $B^{N(d+1) \cdot \ell} 2^{(d+1) \cdot (\ell-1)}$, being generated by the basis $\mathbf{I}_{d+1} \otimes \mathbf{L}$, with

$$\mathbf{L} = \begin{array}{c} \overbrace{\hspace{1.5cm}}^N \quad \overbrace{\hspace{3.5cm}}^{N(\ell-1)} \\ \left[\begin{array}{cccc|cccc|cccc|cccc} B & 0 & \dots & 0 & -2 & -1 & \dots & 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & B & \ddots & \vdots & 0 & -1 & \ddots & \vdots & \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \vdots & \ddots & \ddots & 0 & \vdots & \ddots & \ddots & -1 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ 0 & \dots & 0 & B & 0 & \dots & 0 & -1 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 \\ \hline 0 & 0 & \dots & 0 & 2B & B & \dots & 0 & \ddots & 0 & 0 & \dots & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 & B & \ddots & \vdots & \ddots & 0 & 0 & \ddots & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \ddots & \ddots & B & \ddots & \vdots & \ddots & \ddots & 0 & 0 & \dots & 0 & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & B & \ddots & 0 & \dots & 0 & 0 & 0 & \dots & 0 & 0 \\ \hline \vdots & \vdots & \dots & \vdots & \vdots & \vdots & \dots & \vdots & \ddots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \hline 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 2B & B & \dots & 0 & -2 & -1 & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & B & \ddots & \vdots & 0 & -1 & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & \vdots & \ddots & \ddots & B & 0 & 0 & \ddots & -1 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & \dots & B & 0 & 0 & \dots & -1 \\ \hline 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & \dots & 0 & 2B & B & \dots & 0 \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & \dots & 0 & 0 & B & \ddots & \vdots \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & \dots & 0 & \vdots & \ddots & \ddots & B \\ 0 & 0 & \dots & 0 & 0 & 0 & \dots & 0 & \dots & 0 & 0 & \dots & 0 & 0 & 0 & \dots & B \end{array} \right] \end{array}$$

For any $\mathbf{v} \in (\Lambda^\perp(\mathbf{G}) + \mathbf{c}) \bmod (X+1)$, if the conditions of Lemma 19 are met, we have:

$$\begin{aligned} \left(\mathcal{D}_{\Lambda^\perp(\mathbf{G})+\mathbf{c},r} \bmod (X+1) \right) (\mathbf{v}) &= \frac{\rho_r(\mathbf{c} + \mathcal{L})}{\rho_r(\Lambda^\perp(\mathbf{G}) + \mathbf{c})} \\ &\leq \frac{(1+\varepsilon)r^{(d+1)\cdot\ell} \det(\Lambda^\perp(\mathbf{G}))}{(1-\varepsilon)r^{(d+1)\cdot\ell} \det(\mathcal{L})} \\ &\leq \frac{(1+\varepsilon)}{(1-\varepsilon)} 2^{-(d+1)\cdot(\ell-1)} \end{aligned}$$

Giving us the following bound on the distance to uniformity of $(\mathbf{A}, \mathbf{x}^t \cdot \mathbf{A})$:

$$\begin{aligned} &\frac{1}{2} \sqrt{\sum_{t=1}^{Nk} 2^{dt} \frac{(1+\varepsilon)}{(1-\varepsilon)} 2^{-(d+1)\cdot(\ell-1)+Nk}} \\ &= \frac{1}{2} \sqrt{2^d \frac{1-2^{dNk}}{1-2^d} \frac{(1+\varepsilon)}{(1-\varepsilon)} 2^{-(d+1)\cdot(\ell-1)+Nk}} \end{aligned}$$

This distance can be made as small as desired by setting the parameters correctly. For example, the following bound on ℓ ensures that this quantity is smaller than ε_ℓ .

$$\ell > \frac{1}{d+1} \left(2d - 1 + \log \left(\frac{1-2^{dNk}}{1-2^d} \right) + \log \left(\frac{1+\varepsilon}{1-\varepsilon} \right) + Nk - 2 \log(\varepsilon_\ell) \right)$$

In order to satisfy the conditions of Lemma 19, we need the Gaussian parameter r to be bigger than the smoothing parameters of both $\Lambda^\perp(\mathbf{G})$ and $\mathcal{L}$. We conclude by showing an upper bound of $\eta_\varepsilon(\mathcal{L})$, the bigger one. The columns of $\mathbf{I}_{d+1} \otimes \mathbf{L}$ all have their norm bounded by $2\sqrt{1+B^2}$.

Using Lemma 2, it is sufficient to have

$$r \geq 2 \sqrt{(1+B^2) \cdot \frac{\ln(2N(d+1) \cdot \ell(1+1/\varepsilon))}{\pi}}$$

D Detailed adaptation of [KLSS23, Lemma 7]

We give the proof of Lemma 12. For the sake of readability, we first recall the lemma we prove:

Lemma 20. *For any $\tilde{\mathbf{s}} \in R^d, \mathbf{e} \in \hat{R}_q^{d'}$, let $\tilde{\mathbf{S}}_i$ be the matrix such that $\text{pow}_X \cdot \tilde{\mathbf{S}}_i = \text{pow}_X \otimes \tilde{s}_i$, and $\mathbf{E}_i$ be the matrix such that $\text{pow}_X \cdot \mathbf{E}_i = \text{pow}_X \otimes e_i$. We define $\Gamma = \left(q\mathbf{E}_1^t \mid \dots \mid q\mathbf{E}_{d'}^t \mid -\tilde{\mathbf{S}}_1^t \mid \dots \mid -\tilde{\mathbf{S}}_d^t \right)$. Note that $\text{pow}_X \cdot \Gamma = \text{pow}_X \otimes (q\mathbf{e}^t \mid -\tilde{\mathbf{s}}^t)$. We note*

$$\Sigma = \begin{pmatrix} 2\pi\vartheta_{\mathbf{r}} \mathbf{I}_{d'N} & \mathbf{0} \\ \mathbf{0} & 2\pi\vartheta_{\mathbf{e}'} \mathbf{I}_{dN} \end{pmatrix}, \quad \Sigma_{\mathbf{v}} = \left(\Sigma^{-1} + \frac{1}{2\pi\vartheta_{\mathbf{e}''}} \Gamma^t \Gamma \right)^{-1}.$$

$$\Delta \left((\mathbf{v}, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e''), (\mathbf{v}_0 + \mathbf{v}^*, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'') \right) = 0,$$

where $\mathbf{r} \leftarrow \mathcal{D}_{R^{1 \times d'}, q\sqrt{2\pi\vartheta_{\mathbf{r}}}}$, $\mathbf{e}' \leftarrow \mathcal{D}_{\frac{1}{q}R^{1 \times d}, \sqrt{2\pi\vartheta_{\mathbf{e}'}}}$, $e'' \leftarrow \mathcal{D}_{\frac{1}{q}R, \sqrt{2\pi\vartheta_{\mathbf{e}''}}}$, $\mathbf{v} = \begin{pmatrix} \frac{1}{q}\mathbf{r}^t \\ \mathbf{e}'^t \end{pmatrix}$, $\mathbf{v}_0 = \frac{1}{2\pi\vartheta_{\mathbf{e}''}} \Sigma_{\mathbf{v}} \begin{pmatrix} q\mathbf{e} \\ -\tilde{\mathbf{s}} \end{pmatrix} (\mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'')$ and $\mathbf{v}^* \leftarrow \mathcal{D}_{-\mathbf{v}_0 + \frac{1}{q}R^{d'+d}, \sqrt{\Sigma_{\mathbf{v}}}}$.

Proof. The strategy of our proof is to show that the probability that $(\mathbf{v}, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'')$ is equal to some value $(\mathbf{v}_0 + \mathbf{x}, \mathbf{y})$ is the same as the probability that $(\mathbf{v}_0 + \mathbf{v}^*, \mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'')$ is equal to the same value $(\mathbf{v}_0 + \mathbf{x}, \mathbf{y})$, where $\mathbf{x}$ and $\mathbf{y}$ are dummy variables that are used to express the probabilities of outcome for our random variables.

To compute the probability that $\mathbf{v}$ is equal to $\mathbf{v}_0 + \mathbf{x}$ and that $(\mathbf{r}\mathbf{e} - \mathbf{e}'\tilde{\mathbf{s}} + e'')$ is equal to $\mathbf{y}$, we compute the probability that e'' is equal to $(\mathbf{y} - \mathbf{r}\mathbf{e} + \mathbf{e}'\tilde{\mathbf{s}})$. As $\mathbf{v}$ and e'' are independent, the joint probability can be computed as a product. For given $y \in \hat{R}_q$ and $\mathbf{x} \in \frac{1}{q}R^{d'+d}$, the probability mass assigned to $(\mathbf{v}_0 + \mathbf{x}, \mathbf{y})$, with $\mathbf{v}_0 = \frac{1}{2\pi\vartheta_{e''}}\Sigma_{\mathbf{v}}(\frac{q\mathbf{e}}{-\tilde{\mathbf{s}}})\mathbf{y}$ by the first distribution is:

$$\begin{aligned}
& \mathcal{D}_{\frac{1}{q}\mathbb{Z}^{(d'+d)N}, \sqrt{\Sigma}}(\mathbf{v}_0 + \mathbf{x}) \cdot \mathcal{D}_{\frac{1}{q}\mathbb{Z}^N, \sqrt{2\pi\vartheta_{e''}}}(\mathbf{y} - \Gamma(\mathbf{v}_0 + \mathbf{x})) \\
& \propto \exp\left(-\pi\left((\mathbf{v}_0 + \mathbf{x})^t \Sigma^{-1}(\mathbf{v}_0 + \mathbf{x}) + \frac{1}{2\pi\vartheta_{e''}}(\mathbf{y} - \Gamma(\mathbf{v}_0 + \mathbf{x}))^t(\mathbf{y} - \Gamma(\mathbf{v}_0 + \mathbf{x}))\right)\right) \\
& = \exp\left(-\pi\left((\mathbf{v}_0 + \mathbf{x})^t \Sigma_{\mathbf{v}}^{-1}(\mathbf{v}_0 + \mathbf{x}) + \frac{1}{2\pi\vartheta_{e''}}(\mathbf{y}^t \mathbf{y} - (\mathbf{v}_0 + \mathbf{x})^t \Gamma^t \mathbf{y} - \mathbf{y}^t \Gamma(\mathbf{v}_0 + \mathbf{x}))\right)\right) \\
& = \exp\left(-\pi\left((\mathbf{v}_0 + \mathbf{x})^t \Sigma_{\mathbf{v}}^{-1}(\mathbf{v}_0 + \mathbf{x}) + \frac{1}{2\pi\vartheta_{e''}}\mathbf{y}^t \mathbf{y} - (\mathbf{v}_0 + \mathbf{x})^t \Sigma_{\mathbf{v}}^{-1} \mathbf{v}_0 - \mathbf{v}_0^t \Sigma_{\mathbf{v}}^{-1}(\mathbf{v}_0 + \mathbf{x})\right)\right) \\
& = \exp\left(-\pi\left(\mathbf{x}^t \Sigma_{\mathbf{v}}^{-1} \mathbf{x} - \mathbf{v}_0^t \Sigma_{\mathbf{v}}^{-1} \mathbf{v}_0 + \frac{1}{2\pi\vartheta_{e''}}\mathbf{y}^t \mathbf{y}\right)\right)
\end{aligned}$$

So, for any given y , the mass given to $\mathbf{v}_0 + \mathbf{x}$ is proportional to $\rho_{\sqrt{\Sigma_{\mathbf{v}}}}$. Hence the result.